

The Computational Power of Population Protocols

Dana Angluin
Yale University

James Aspnes*
Yale University

David Eisenstat
Princeton University

Eric Ruppert†
York University

May 24, 2022

Abstract

We consider the model of population protocols introduced by Angluin *et al.* [AAD⁺04], in which anonymous finite-state agents stably compute a predicate of the multiset of their inputs via two-way interactions in the all-pairs family of communication networks. We prove that all predicates stably computable in this model (and certain generalizations of it) are semilinear, answering a central open question about the power of the model. Removing the assumption of two-way interaction, we also consider several variants of the model in which agents communicate by anonymous message-passing where the recipient of each message is chosen by an adversary and the sender is not identified to the recipient. These one-way models are distinguished by whether messages are delivered immediately or after a delay, whether a sender can record that it has sent a message, and whether a recipient can queue incoming messages, refusing to accept new messages until it has had a chance to send out messages of its own. We characterize the classes of predicates stably computable in each of these one-way models using natural subclasses of the semilinear predicates.

1 Introduction

In 2004, Angluin *et al.* [AAD⁺04] proposed a new model of distributed computation by very limited agents called a population protocol. In this model, finite-state agents interact in pairs chosen by an adversary, with both agents updating their state according to a joint transition function. For each such transition function, the resulting population protocol is said to stably compute a predicate on the initial states of the agents if, after sufficiently many interactions in a fair execution, all agents converge to having the correct value of the predicate. Motivating scenarios include models of the propagation of trust in populations of agents [DF01] and interactions of passively mobile sensors [AAD⁺04, AAD⁺06]. Similar models of pairwise interaction have been used to study the propagation of diseases [Bai75] and rumors [DK65] in human populations and to justify the Chemical Master Equation [Gil92], suggesting that the model of population protocols may be fundamental in several fields.

Because the agents in a population protocol have only a constant number of states, it is impossible for them to adopt distinct identities, making them effectively anonymous. An agent encountering another agent cannot tell in general whether it has interacted with that agent before. Despite these limitations, populations of such agents can compute surprisingly powerful predicates on their initial states under a reasonable global fairness condition. When each agent may interact with every other agent, any predicate over the counts of initial states definable in Presburger arithmetic is computable [AAD⁺04, AAD⁺06]. When each agent

*Supported in part by NSF grants CNS-0305258 and CNS-0435201.

†Supported in part by the Natural Sciences and Engineering Research Council of Canada.

has only a bounded set of neighbors with which it can interact, linear-space computable predicates are computable [AAC⁺05].

In this paper we give exact characterizations of the class of stably computable predicates in the all-pairs interaction graph for the original population protocol model and certain natural variants of it that we introduce to model the restrictions of one-way communication. Most of the results in this paper have appeared in extended abstract form in the two conference papers [AAE06b, AAER05]. In Section 7 we also correct an erroneous claim in [AAER05].

1.1 Stably Computable Predicates Are Semilinear

Angluin *et al.* [AAD⁺04, AAD⁺06] showed that various common predicates such as parity of the number of agents, whether agents in some initial state a outnumber agents in another initial state b , and so forth, could be stably computed by simple population protocols. They further showed that population protocols could in fact compute any semilinear predicate, which are precisely those predicates definable in first-order Presburger arithmetic [Pre29]. But it was not known whether there were other, stronger predicates that could also be computed by a population protocol, at least in the simplest case where every agent was allowed to interact with every other agent. We show that this is not the case: that the semilinear predicates are precisely the predicates that can be computed by a population protocol if there is no restriction on which pairs of agents can interact with each other. This gives an exact characterization of the predicates stably computable by population protocols, answering the major open question of [AAD⁺04, AAD⁺06].

These results also answer an open question in [AAC⁺05]: whether requiring population protocols to deal with stabilizing inputs (rather than assuming all inputs are available at the start of computation) reduces their computational power. We show that the answer is no; in both cases, the stably computable predicates are precisely the semilinear predicates.

The semilinearity theorem is proved for a generalization of the model of population protocols, and applies to stable computation in other models that have the property that a partitioned subpopulation can still run on its own (in the sense that agents do not have any mechanism to detect if additional agents are present but not interacting). Other models with this property include vector addition systems [HP78], some forms of Petri nets, and 1-cell catalytic P-systems. The relationship between semilinear sets and catalytic P-systems, which can be represented as population protocols in which all state-changing interactions occur between an unmodified catalyst and at most one non-catalyst agent, has previously been considered by Ibarra, Dang and Egencioglu [IDE04], who show that the sets accepted by 1-cell catalytic P-systems are semilinear. Our results extend this fact to predicates stably computable in this model, even without the catalytic restriction.

Semilinearity is strongly tied to the notion of stable computation, in which a correct and stable output configuration must always be reachable at any step of the computation. Mere reachability of a desired final configuration is not enough: the reachability sets of population protocols are not in general semilinear. An example of this phenomenon may be derived from the construction given by Hopcroft and Pansiot of a non-semilinear reachability set for a six-dimensional vector addition system [HP78].

1.2 One-way communication

We also introduce variants of the population protocol model that use forms of one-way communication analogous to traditional asynchronous message-passing models, and exactly characterize their computational power in the all-pairs communication graph in terms of natural subclasses of the semilinear predicates. In the one-way models, pairwise interactions are split into separate send and receive events that each affect at most a single agent. These models may better reflect communication in the context of sensor networks,

where radio communication may not be bidirectional, even between nearby sensors. Moreover, one-way message-passing primitives may be easier to implement in practice.

For the one-way models, we consider the following attributes. The sender may be allowed to change its state as a result of sending a message (transmission models), or not (observation models). The send and receive events for a message may occur simultaneously (immediate delivery models) or may be subject to a variable delay (delayed delivery and queued delivery models). In the queued delivery model, a receiver may choose to postpone incoming messages until it has had a chance to send a message of its own; in the simpler delayed delivery models, the receiver does not have this option. Thus, we consider five one-way models: immediate and delayed observation, immediate and delayed transmission, and queued transmission.

The immediate models are perhaps best understood in terms of interactions between agents, while the delayed and queued models are better thought of in terms of messages sent from one agent to another. In the transmission models, the sender is aware that it has sent a message, and may update its state accordingly. In the observation models, the sender is unaware of being observed by the receiver, and therefore does not update its state. For example, consider a passive radio frequency tag that does not update its state in response to being read; this is an example of immediate observation, in which the message (tag data) is immediately delivered (read), but the state of the sender is unchanged. A web page that updates a counter in response to a visit can be thought of as an example of immediate transmission: the message (web page contents) is immediately delivered (viewed), and the web page is aware of the transaction and updates its state accordingly.

We give exact characterizations of the classes of predicates that can be stably computed in each of the one-way models we consider. We show that the model of queued transmission stably computes exactly the semilinear predicates and is equivalent in power to the standard population protocol model with two-way interactions. The other models are strictly weaker, and can be characterized in terms of natural subclasses of the semilinear predicates. Weakest of all is the delayed observation model, which allows only the detection of the presence or absence of each possible input symbol. Next is the immediate observation model, which allows counting of input symbols up to an arbitrary constant limit. The immediate and delayed transmission models are equivalent to each other in power, and add the ability to count input symbols modulo an arbitrary constant. These results give us a precise and detailed understanding of the capabilities of the one-way models.

2 Related Work

2.1 Population Protocols and Related Models

Stable computation by population protocols was introduced in [AAD⁺04, AAD⁺06]. It was shown that all semilinear predicates are stably computable by population protocols in the all-pairs interaction graph. It was also shown that the all-pairs interaction graph has the least computational power, in the sense that it may be simulated in any other connected interaction graph. This work was inspired by models of the propagation of trust studied by Diamadi and Fischer [DF01]. Related models of automata with a central finite control and storage consisting of an unordered multiset of tokens was studied in [AAD⁺03].

Angluin *et al.* [AAD⁺04, AAD⁺06] also introduced probabilistic population protocols, in which a uniform random choice of pairs to interact replaces the fairness condition; this enables quantification of the probability of error and the number of interactions to convergence as a function of n , the number of agents in the population. It was shown that each semilinear predicate can be computed in an expected $O(n^2 \log n)$ interactions, and that a register machine with a constant number of registers with $O(\log n)$

bits per register could be simulated with inverse polynomial error probability and polynomial slowdown. Recent work [AAE06a] has given faster protocols for these problems under the assumption that a unique leader is present in the population: each semilinear predicate can be computed in an expected $O(n \log^4 n)$ interactions, and an improved register machine simulation can be done with inverse polynomial error in $O(n \text{polylog}(n))$ interactions per step.

The question of what properties of the interaction graph are stably computable by population protocols was studied in [AAC⁺05]. It was shown that for any d , there is a population protocol that organizes any connected interaction graph of maximum degree at most d into a linear memory of $\Theta(n)$ bits, which is asymptotically optimal. In the same paper, the population protocol model was extended to stabilizing inputs, in which each agent has an input that may change finitely many times over the course of the computation before stabilizing to a final value; this permits composition of protocols, in which the stabilizing outputs of one protocol become the stabilizing inputs of another protocol. It was shown that all the semilinear predicates can be computed with stabilizing inputs in the all-pairs interaction graph. Thus, the present paper shows that the precisely the same predicates are stably computable with or without stabilizing inputs in the all-pairs interaction graph.

Self-stabilizing population protocols were studied in [AAFJ05], which gives self-stabilizing protocols for token circulation in a directed ring, directing an undirected ring, local addressing in a degree bounded interaction graph, and leader election in a ring with a constant bound k on the smallest nondivisor of the ring size. The question of resilience to crash faults and transient faults was considered by Delporte-Gallet *et al.* [DGFG06], who give a general method to transform a population protocol into one that can withstand up to c crash faults and t transient faults.

2.2 Comparison with Asynchronous Message-passing

In an asynchronous message-passing model, agents communicate by sending messages. An agent may spontaneously send a message at any time, which is delivered to a recipient at some later time. The recipient may respond to the message by updating its state and possibly sending one or more messages. In the standard asynchronous model, senders can choose the recipients of their messages, and recipients are aware of the identities of the senders of messages they receive; however, in the population protocol models we consider, these assumptions are dropped.

Agents in the population protocol models are assumed to be finite-state. Moreover, algorithms in this model are uniform: the description of the protocol cannot depend on the number of agents in the system. Together with a transition rule that depends only on the states of the two interacting agents, these assumptions naturally yield a model in which agents are effectively anonymous. In this respect, the population protocol models are weaker than a typical message-passing model, where processes have identities. In addition, not only does a receiver not learn the identity of the sender, but a sender cannot direct its message to a particular receiver. This is unusual even in anonymous message-passing models, which typically assume that a process can use some sort of local addressing to direct messages to specific neighbors.

The question of what computations can be performed in anonymous systems, where processes start with the same state and the same programming, has a long history in theoretical distributed computing. Many early impossibility results such as [Ang80] assume both anonymity and symmetry in the communication model, which limits what can be done without some mechanism for symmetry-breaking. See [FR03] for a survey of many such impossibility results. More recent work targeted specifically at anonymity has studied what problems are solvable in message-passing systems under various assumptions about the initial knowledge of the processes [BV99, BV01, Sak99], or in anonymous shared-memory systems where the properties of the supplied shared objects can often (but not always, depending on the details of the model)

be used to break symmetry and assign identities [ASS02, AGM02, BPSV06, ES94, GR05, JT90, KOPS00, LP90, PPTV98, Ten90]. This work has typically assumed few limits on the power of the processes in the system other than the symmetry imposed by the model.

Asynchronous message-passing systems may be vulnerable to a variety of failures, including failures at processes such as crashes or Byzantine faults, and failures in the message delivery system such as dropped or duplicated messages. In this paper we assume fault-free executions; however, Delporte-Gallet *et al.* [DGFG06] characterize the power of population protocols in the presence of crash faults and transient faults.

Because message delivery is asynchronous, making any sort of progress requires adopting some kind of fairness condition to exclude executions in which indefinitely-postponed delivery becomes equivalent to no delivery. A minimal fairness condition might be that if some process sends a particular message m infinitely often, then each other process receives the same message m infinitely often. In Section 15, we show that even with unbounded states and message lengths, this minimal fairness condition provides only enough power to detect the presence or absence of each possible input because of the very strong anonymity properties of the model. Instead we adopt a stronger global fairness condition derived from that used in [AAD⁺04, AAD⁺06], which is defined and further discussed in Section 4.

Considering the communication capabilities of the population protocol models, the two-way population protocol model is stronger than a typical message-passing model: communication between two interacting agents is instantaneous and bidirectional. Instantaneous communication is also a feature of the immediate transmission and immediate observation models. The observation models are weaker than message-passing in that the sender is not aware of sending a message. In the immediate and delayed transmission models, an agent must be prepared to receive and react to a message at all times. In the queued transmission model, which most closely approximates asynchronous message-passing, an agent may postpone receiving a message until it has sent messages of its own, and we show that this capability is essential to achieve the full power of the model.

3 Preliminaries

Let $\mathbb{N}$ denote the set of natural numbers, $0, 1, 2, \dots$. The set of all functions from a set X to a set Y is denoted Y^X . Let E be a finite nonempty set. For all $f, g \in \mathbb{R}^E$, we define the usual vector space operations

$$\begin{aligned} (f + g)(e) &:= f(e) + g(e) & \forall e \in E \\ (f - g)(e) &:= f(e) - g(e) & \forall e \in E \\ (cf)(e) &:= cf(e) & \forall c \in \mathbb{R}, e \in E \\ f \cdot g &:= \sum_{e \in E} f(e)g(e). \end{aligned}$$

Abusing notation, we define a 0 vector and standard basis vectors

$$\begin{aligned} 0(e) &:= 0 & \forall e \in E \\ e(e') &:= [e = e'] & \forall e, e' \in E, \end{aligned}$$

where $[condition]$ is 1 if *condition* is true and 0 otherwise. We define a natural partial order on $\mathbb{R}^E$ componentwise:

$$f \leq g \Leftrightarrow (\forall e \in E) f(e) \leq g(e).$$

Next we define the set of **populations** on E :

$$\text{Pop}(E) := \mathbb{N}^E \setminus \{0\}.$$

These may be interpreted as the nonempty multisets on E : for any $f \in \text{Pop}(E)$ and $e \in E$, $f(e)$ represents the multiplicity of the element e in the multiset represented by f . Then, the partial order $\leq$ corresponds to the subset order on multisets.

4 A Unified Framework

In each of the models we consider, a **protocol** can be considered as determining five components: a countable set $\mathcal{C}$ of **configurations**; a set of **input symbols** Σ ; a binary relation $\rightarrow$ on $\mathcal{C}$ that captures when the first configuration **can reach** the second in one step; a function $I : \text{Pop}(\Sigma) \rightarrow \mathcal{C}$ that takes inputs to **initial** configurations; and a partial function $O : \mathcal{C} \rightarrow \{0, 1\}$ that gives the **output** of each configuration on which it is defined. We take $\rightarrow^*$ to be the reflexive-transitive closure of $\rightarrow$. We say c' is **reachable** from c if $c \rightarrow^* c'$. In this unified framework, we make the following definitions.

A configuration c is **output stable with output** b if $O(c) = b$ and for every d such that $c \rightarrow^* d$, $O(d) = b$. We define $\mathcal{S}_b$ to be the set of configurations that are output stable with output b , for $b \in \{0, 1\}$, and define $\mathcal{S} = \mathcal{S}_0 \cup \mathcal{S}_1$ to be the set of all **output stable** configurations. Thus a configuration is output stable if and only if its output is defined, and every configuration reachable from it has the same defined output.

An **execution** is a (finite or infinite) sequence of configurations $c_0, c_1, \dots$ such that for all j , we have $c_j \rightarrow c_{j+1}$. An execution $c_0, c_1, \dots$ is **fair** if for all $c' \in \mathcal{C}$, either there exist infinitely many j such that $c_j = c'$ or there exists j such that $c_j \not\rightarrow^* c'$. This is equivalent to the global fairness condition defined in [AAD⁺04, AAD⁺06], and implies that a global configuration that is infinitely often reachable in a fair execution must occur infinitely often in that execution. This fairness condition may be viewed as an attempt to capture useful probability 1 properties in a probability-free model. In Section 15 we show that a weaker, but plausible, fairness condition severely limits the power of the model. Since $\mathcal{C}$ is countable, any finite execution is a prefix of a fair execution, which can be constructed as follows. Fix an enumeration of $\mathcal{C}$ where each configuration appears infinitely often. Then, starting with the finite execution, repeatedly extend it with a sequence of configurations that reaches the next configuration in the enumeration that is reachable from the last configuration of the execution constructed in the previous step.

A fair execution $c_0, c_1, \dots$ **converges with output** b if there exists an m such that for all $j \geq m$, the function O is defined on c_j and $O(c_j) = b$. In particular, a fair execution converges with output b if and only if it reaches an output stable configuration with output b . A protocol is **well-specified** if, for all initial configurations $c_0 \in I(\text{Pop}(\Sigma))$, there exists a b such that all fair executions $c_0, c_1, \dots$ converge with output b . In a well-specified protocol, every fair execution starting from an input configuration converges with an output that is determined by that input configuration.

A well-specified protocol induces a predicate $\psi : \text{Pop}(\Sigma) \rightarrow \{0, 1\}$. We say that this protocol **stably computes** ψ . The results in this paper characterize the predicates ψ stably computable in various models of finite local state distributed computing.

5 Definitions of Models

In this section we define the standard model of two-way population protocols in the all-pairs interaction graph and the one-way variants we consider, as well as the Abstract Model, which subsumes all of these.

For each model, we specify the configurations, input alphabet, one-step reachability relation, input map and output map required by the unified framework above.

5.1 Two-way

A **two-way** protocol is specified by five components: Q , a finite set of states; Σ , a finite set of input symbols; $\delta : Q \times Q \rightarrow Q \times Q$, a joint transition function; $\iota : \Sigma \rightarrow Q$, the initial state mapping; and $o : Q \rightarrow \{0, 1\}$, the individual output function.

We define

$$\begin{aligned}\mathcal{C} &:= \text{Pop}(Q) \\ I(x) &:= \sum_{\sigma \in \Sigma} x(\sigma) \iota(\sigma) \\ O(c) &:= b \text{ if for all } q \in Q, c(q) \geq 1 \Rightarrow o(q) = b\end{aligned}$$

We define $c \rightarrow c'$ if $q_1 + q_2 \leq c$, $c' = c - q_1 - q_2 + q'_1 + q'_2$ and $\delta(q_1, q_2) = (q'_1, q'_2)$.

A configuration in this model is a multiset that gives the states of all the agents. Because agents do not have identifiers and we are considering the all-pairs communication graph, agents in the same state are interchangeable; thus, the multiset of their states completely specifies the global state of the population. An initial configuration sets the state of each agent according to the input symbol assigned to it. A step is an interaction between two agents and simultaneously updates both of their states according to the value of the joint transition function of their current states. If the transition used is $\delta(q_1, q_2)$, we refer to the agent in state q_1 as the **initiator** and the other agent as the **responder**.

Each agent has an individual output of 0 or 1 determined by its current state via the function o . The configuration output function is 0 (respectively, 1) if all the individual outputs are 0 (respectively, 1). If the individual outputs are mixed 0's and 1's, then the configuration output function is undefined.

5.2 Transmission with Queuing

A **queued transmission** protocol is specified by seven components: Q , a finite set of states; M , a finite set of messages that is disjoint from Q ; Σ , a finite set of input symbols; $\delta_s : Q \rightarrow M \times Q$, a transition function for sent messages; $\delta_r : Q \times M \rightarrow Q$, a partial transition function for received messages; $\iota : \Sigma \rightarrow Q$, the initial state function; and $o : Q \rightarrow \{0, 1\}$, the individual output function. We define

$$\begin{aligned}\mathcal{C} &:= \{c \in \mathbb{N}^{Q \cup M} : c(q) > 0 \text{ for some } q \in Q\} \\ I(x) &:= \sum_{\sigma \in \Sigma} x(\sigma) \iota(\sigma) \\ O(c) &:= b \text{ if for all } q \in Q, c(q) \geq 1 \Rightarrow o(q) = b.\end{aligned}$$

We have $c \rightarrow c'$ if for some state $q \in c$ and some message m , $c' = c - q + q' + m$ where $(m, q') = \delta_s(q)$; or if there exists a message $m \in c$ and a state $q \in c$ such that $c' = c - q - m + \delta_r(q, m)$. In the latter case, $\delta_r(q, m)$ must be defined.

A configuration in this model is the multiset of all agents' states and all messages in transit. (They cannot be confused, because Q and M are disjoint.) An input configuration has no messages in transit and sets the state of each agent according to the input symbol assigned to it. A step is either a send event, in which an agent in state q adds a message m to the multiset of messages in transit and goes to state q' (according to

$\delta_s(q) = (m, q')$,) or a receive event, in which a message m is removed from the multiset of messages in transit and is delivered to an agent in state q , which updates its state to q' (according to $\delta_r(q, m) = q'$.) If $\delta_r(q, m)$ is not defined, then message m cannot be delivered to an agent in state q ; this permits agents to refuse to receive messages temporarily. (We note that because an agent in an observation model does not change state upon being observed, there would be no way for it to leave a non-receive enabled state; thus we do not consider a queued observation model.)

5.3 Immediate Transmission and Observation

Immediate transmission is a special case of the two-way model in which the state of the initiator is updated independent of the state of the responder. That is, there exist functions $\delta_1 : Q \rightarrow Q$ and $\delta_2 : Q \times Q \rightarrow Q$ such that for all $q_1, q_2 \in Q$, we have $\delta(q_1, q_2) = (\delta_1(q_1), \delta_2(q_1, q_2))$. Thus, the initiator (or sender) is aware of the fact that an interaction has taken place (or of sending a message), and may update its state accordingly, but it is not aware of the state of the responder (or receiver.)

Immediate observation is a special case of immediate transmission in which δ_1 is the identity function. This is the situation in which the initiator (or sender) is not aware of being “observed” by the responder (or recipient) and therefore does not have an opportunity to update its state.

5.4 Delayed Transmission and Observation

Delayed transmission is a special case of queued transmission, with the requirement that δ_r be a total function. In this case, the recipient does not have the option of temporarily refusing to receive messages, which means that it is in danger of being “overwhelmed” by incoming messages. Our characterization results show that this indeed limits the power of protocols in this model.

Delayed observation is a special case of delayed transmission in which for all $q \in Q$, we have $\delta_s(q) = (q, m)$ for some $m \in M$. In this model, the weakest of those we consider, an agent can neither refuse incoming messages nor update its state when it has sent a message.

5.5 The Abstract Model

In order to present our semilinearity characterization in full generality, we introduce another model. All of the preceding single step rules can be described in terms of replacing one collection of elements (states or messages) with another without regard to the other elements in the configuration. In this model it is also convenient to identify input symbols with the states they map to, dispensing with the need for an initial state map ι .

E is a set of elements, which may be either states or messages; $\Sigma \subseteq E$ is a set of input symbols; $\rightarrow$ is a relation on $\text{Pop}(E)$ such that if $c \rightarrow c'$, then for all $d \in \text{Pop}(E)$, we have $c + d \rightarrow c' + d$; $o : E \rightarrow \{0, 1\}$ is the individual output map. Then,

$$\begin{aligned} \mathcal{C} &:= \text{Pop}(E) \\ I(x) &:= x, \text{ and} \\ O(c) &:= b \text{ if for all } e \in E, c(e) \geq 1 \Rightarrow o(e) = b. \end{aligned}$$

To see that this generalizes the two-way model, we take E to be the disjoint union of the input symbols and states of the two-way model, and treat the input symbol σ as equivalent to the state $\iota(\sigma)$, extending the individual output map o to Σ by $o(\sigma) = o(\iota(\sigma))$. To see that the Abstract Model also generalizes the queued

transmission model, we define E to be the disjoint union of the input symbols, states, and two copies of the messages of the queued transmission model. Having two copies of each message m allows us to designate one copy as having output 0 and the other as having output 1. Again we treat input symbol σ as equivalent to the state $\iota(\sigma)$, and extend the individual output map o to Σ by $o(\sigma) = o(\iota(\sigma))$. Also, we extend o to the two copies of each message by defining it to be the designated output of that copy. To ensure that outputs propagate from agent states to messages, we add rules that take a state of an agent and a copy of a message, and change (if necessary) the designated output of the message to be the same as the output of the state. Each send rule is modified to send a message with the same output value as the sender, and the receive rules ignore the output values of the messages. These changes guarantee that when the outputs of the agents stabilize, the designated outputs of the messages stabilize to the same thing. Thus, every predicate stably computable in either the two-way or the queued transmission model is stably computable in the Abstract Model.

5.6 Mirrors and Messages to Self

Separating message transmission and receipt creates the possibility that an agent may receive its own message. Because senders are not identified, such an agent will in general not be able to recognize the message as its own. This can be thought of as including **mirrors**, or self-loops in the interaction graph controlling which agents can communicate, which we otherwise take to consist of all ordered pairs of agents. In general, we assume that this does not occur in the two-way and immediate delivery models, which are perhaps best thought of as interaction models, but may occur in the delayed and queued delivery models, on the principle that once an anonymous message is sent it may be delivered to anyone. This has at most a minor effect on the computational power of the models we consider, which we note below as appropriate.

6 Predicate Classes

We now define the classes of predicates used in our characterizations. Although stably computable predicates are defined only on $\text{Pop}(\Sigma)$, our proofs are facilitated by defining predicate classes on $\mathbb{Z}^\Sigma$. The **support** of a predicate is the set of all inputs that make it true.

6.1 Semilinear Predicates

The most important class of predicates we consider is the class of **semilinear** predicates, which we write **SLIN**. This class can be defined in several equivalent ways.

A **linear set** is a set of the form $\{b + k_1 p_1 + k_2 p_2 + \dots + k_n p_n \mid k_1, k_2, \dots, k_n \geq 0\}$, where $b, p_1, p_2, \dots, p_n$ are vectors. The vector b is the **base** of the linear set, and the vectors p_i are the **period** vectors. A **semilinear set** is a finite union of linear sets. A **(semi)linear predicate** is a predicate whose support is (semi)linear predicate.

Semilinear sets are also precisely the sets definable by first-order formulas in **Presburger arithmetic** [Pre29] which are formulas in arithmetic that use only $<, +, 0, 1$, and the standard logical quantifiers and connectives. Here the set consists of all satisfying assignments of the free variables; for example, the semilinear set $S = \{(1, 0) + k_1(1, 0) + k_2(0, 2)\} \cup \{(0, 2) + k_3(2, 0)\}$, depicted in Figure 1, consists precisely of the satisfying assignments (x, y) of the formula

$$(\exists z : (z \geq 0) \wedge (x = z + z + 1) \wedge (y \geq z)) \vee (\exists z : (z \geq 0) \wedge (x = z + z) \wedge (y = 1 + 1)), \quad (1)$$

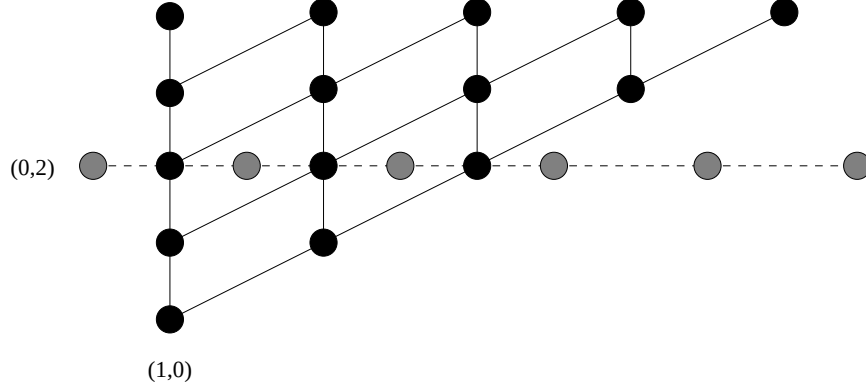


Figure 1: A semilinear set S , equal to the union of the linear set of all points $\{(1, 0) + k_1(1, 0) + k_2(0, 2)\}$ (dark circles) and the linear set $\{(0, 2) + k_3(2, 0)\}$ (shaded circles).

where $x = y$ abbreviates $\neg((x < y) \vee (y < x))$ and $x \geq y$ abbreviates $\neg(y < x)$. It follows immediately from the correspondence between semilinear sets and Presburger formulas that the semilinear sets are closed under complement, finite intersection and finite union. Thus a predicate is semilinear if and only if its complement is semilinear.

A curious and useful property of Presburger formulas is that all quantifiers (and their bound variables) can be eliminated by the addition of binary relations $\equiv_m$ that test for equality modulo m for any nonnegative integer m [Pre29]. For example, the formula (1) defining S can be rewritten without quantifiers as

$$((x \geq 0) \wedge (x \equiv_2 1) \wedge (x \leq y + y + 1)) \vee ((x \geq 0) \wedge (x \equiv_2 0) \wedge (y = 1 + 1)).$$

This yields another characterization: a semilinear predicate is a boolean combination of threshold predicates, whose support takes the form $\{x \mid x \cdot v \geq r\}$ for some $v \in \mathbb{Z}^\Sigma$ and $r \in \mathbb{Z}$, and modulo predicates, whose support takes the form $\{x \mid x \cdot v \equiv r \pmod{m}\}$ for some $v \in \mathbb{Z}^\Sigma$ and $r, m \in \mathbb{Z}$ with $m > 0$. Viewed geometrically, sets of the first type consist of points on one side of a hyperplane, and sets of the second type are lattices. As an example of a predicate of the first type, consider **comparison**, which is true if the number of a 's in the input exceeds the number of b 's in the input. As an example of a predicate of the second type, consider **parity**, which is true if the number of a 's in the input is odd.

In yet another characterization, Parikh's Theorem [Par66] shows that a subset S of $\mathbb{N}^d$ is semilinear if and only if there is a context-free language L over an alphabet of d symbols such that S consists of the vectors of multiplicities of alphabet symbols of strings in L . Moreover, the same statement holds with regular languages in place of context-free languages. Using this characterization, it is not difficult to see that the following predicates on the number of a 's and b 's in the input are not semilinear: the number of a 's is a prime, the number of a 's is a square, the number of a 's is a power of 2, and the number of a 's is bounded above by $\sqrt{2}$ times the number of b 's.

6.2 The Classes MOD and coreMOD

We write **MOD** for the class of boolean combinations of modulo predicates only.

Let $\Sigma' \subseteq \Sigma$ be any nonempty subset of the input alphabet. Then $x \in \mathbb{Z}^\Sigma$ is a k -rich profile with respect to Σ' if $x(\sigma') \geq k$ for all $\sigma' \in \Sigma'$ and $x(\sigma) = 0$ for all $\sigma \in \Sigma \setminus \Sigma'$. Let ψ and ψ' be predicates on $\mathbb{Z}^\Sigma$. If

$\psi(x) = \psi'(x)$ for every x that is a k -rich profile with respect to Σ' , then we say that ψ and ψ' are **k -similar with respect to Σ'** .

The class **coreMOD** is the class of predicates ψ such that for every nonempty $\Sigma' \subseteq \Sigma$, there exists a $\psi' \in \mathbf{MOD}$ and $k \geq 0$ such that ψ is k -similar to ψ' with respect to Σ' . Clearly $\mathbf{MOD} \subseteq \mathbf{coreMOD}$, so the parity predicate is in **coreMOD**. The comparison predicate is not in **coreMOD** because it is not k -similar to any modulo predicate with respect to $\{a, b\}$ for any $k \geq 0$. However, if we consider the predicate over the alphabet $\{a, b, c\}$ that is true when there is exactly one c and the number of a 's exceeds the number of b 's, then this predicate is in **coreMOD**. To see this, note that the predicate is false when the number of c 's is 0 or at least 2, so $k = 2$ suffices as a witness for any nonempty subalphabet.

6.3 Simple Threshold Predicates

We define a **simple threshold predicate** to be a threshold predicate with support $\{x \mid x \cdot v \geq r\}$ in which $v = \sigma$ for some input symbol σ . An example of a simple threshold predicate is one that is true when the number of a 's is at least 5. Then we define **COUNT_k** to be the class of boolean combinations of simple threshold predicates in which the threshold value $r \leq k$. A predicate in **COUNT_k** is completely determined by the counts of the input symbols truncated at k . For example, when $k = 1$, such a predicate depends only on the presence or absence of each input symbol. Finally, **COUNT_{*}** is the union of the classes **COUNT_k** for $k = 1, 2, 3, \dots$. The comparison predicate is an example of a threshold predicate that is not in **COUNT_{*}**.

7 Summary of Characterizations

The computational power of the population protocol models we consider is summarized in Figure 2. For each model we give the class of predicates on $\text{Pop}(\Sigma)$ that can be stably computed by protocols in the model. Protocols in the Abstract Model, which subsumes the two-way and queued transmission models, stably compute exactly the semilinear predicates. The immediate and delayed transmission models are equal in power, and stably compute exactly those semilinear predicates that are in **coreMOD**; they cannot stably compute the comparison predicate. Immediate observation protocols stably compute exactly those predicates determined by the counts of the input symbols truncated at k for some k ; they cannot stably compute the comparison predicate or the parity predicate. Delayed observation protocols stably compute exactly those predicates determined by the presence or absence of each input symbol; they cannot stably compute the comparison predicate, the parity predicate or simple threshold predicates for thresholds greater than 1.

The results in [AAER05] incorrectly claimed that any two-way stably computable predicates k -similar with respect to Σ to a predicate in **MOD** is stably computable in the immediate and delayed transmission models, omitting the quantification over all nonempty subalphabets of the input alphabet. To see why the more complex condition is required, consider the predicate over the alphabet $\{a, b, c\}$ that is true if there is *at most one* c in the input and the number of a 's exceeds the number of b 's. This predicate is 2-similar to the constant false predicate with respect to $\{a, b, c\}$ but it is not in the class **coreMOD**, because for no $k \geq 0$ is it k -similar to a predicate in **MOD** with respect to the subalphabet $\{a, b\}$. This predicate is not stably computable in the immediate and delayed transmission models.

Model	Power
Abstract	SLIN
Two-way	SLIN
Queued Transmission	SLIN
Immediate Transmission	SLIN $\cap$ coreMOD
Delayed Transmission	SLIN $\cap$ coreMOD
Immediate Observation	COUNT _*
Delayed Observation	COUNT ₁

Figure 2: The Power of Population Protocols

8 Protocols

We first give protocols for each model in Figure 2 to establish that each model is at least as powerful as claimed.

8.1 The Two-Way Model

From [AAD⁺04, AAD⁺06] we have the following.

Theorem 1. *Every semilinear predicate on $\text{Pop}(\Sigma)$ is stably computable by a two-way population protocol.*

To help us describe protocols in the one-way models, we here give specific two-way protocols for simple threshold, modulo and general threshold predicates.

Simple threshold predicates. The following protocol computes the simple threshold predicate $\psi(x) := [x \cdot \sigma \geq k]$, which is true when there are at least k occurrences of the input symbol σ in the input x .

$$\begin{aligned}
Q &:= \{0, 1, \dots, k\} \\
\delta(q_1, q_2) &:= \begin{cases} (q_1, k) & \text{if } q_1 = k \\ (q_1, q_2 + 1) & \text{if } 1 \leq q_1 < k \text{ and } q_1 = q_2 \\ (q_1, q_2) & \text{otherwise} \end{cases} \\
\iota(\sigma') &:= [\sigma' = \sigma] \\
o(q) &:= [q = k]
\end{aligned}$$

Initially, agents with input σ are in state 1 and all other agents are in state 0. As this protocol runs, the states of the agents make a “tower”. All but one of the agents in state i advance to state $i + 1$ in each case. This tower will extend to k if and only if there are initially at least k agents in nonzero states. We note that this is an immediate observation protocol because the state of the initiator remains unchanged by an interaction.

Active and passive agents. In the protocols for the modulo predicate $\psi(x) = [x \cdot v \equiv r \pmod{m}]$ and the threshold predicate $\psi(x) = [x \cdot v \geq r]$ agents fall into one of two categories: active or passive. Every agent is initially active, and there is at least one active agent at all times. Each active agent also has a data value, initially $\sigma \cdot v$, where σ is the input symbol for this agent. When a passive responder meets an active initiator, it copies the output of the active initiator. When two passive agents meet, nothing happens. When two active agents meet, they attempt to combine their data values, and one of them may become passive.

Modulo predicates. For a modulo predicate, $\psi(x) = [x \cdot v \equiv r \pmod{m}]$, the data values of active agents combine by sum modulo m . The initiator becomes passive while the responder remains active and keeps the combined data value. Eventually, exactly one agent is active and has the correct sum modulo m , and distributes the correct output to all of the other agents. We note that this is an immediate transmission protocol because the state of the initiator is updated uniformly, independent of the state of the responder.

Threshold predicates. The situation for a threshold predicate, $\psi(x) = [x \cdot v \geq r]$, is more complicated. In the end there may be multiple active agents, but they will all agree on the output. We assume without loss of generality that $r \geq 0$. The active states are a range of values that include all possible initial data values and $2r - 1$. The output is 1 if and only if the data value is at least r . Suppose u and v are the data values when two active agents meet. If $u + v$ is representable by an active state, one agent remains active with this data value; the other becomes passive. Otherwise, both agents remain active and they average their data values, that is, one becomes $\lceil \frac{u+v}{2} \rceil$ and the other becomes $\lfloor \frac{u+v}{2} \rfloor$. These transitions maintain the invariant that the sum of all the active agents' data values is $x \cdot v$. Note that this protocol involves updates of both states depending on both states; it is not an immediate transmission protocol.

To show correctness, we distinguish three cases based on the sum of all the initial data values. In the first case, the sum is negative. Eventually, no active agent will have a positive data value. In the second case, the sum is between 0 and $r - 1$. There will eventually be exactly one active agent with this data value. In the third case, the sum is at least r . The number of active agents with data value less than r never increases. The only way it can increase is after an interaction involving an active agent with value at least r where both agents remain active. This will happen, however, only in the case when the two agents average their values, which never happens with an agent with value at least r unless the result is both agents having value at least r . After there are no more agents with negative values, any interaction with an agent whose value is less than r will decrease the number of such agents.

8.2 Queued Transmission

When combined with Theorem 1, the following theorem implies that every semilinear predicate on $\text{Pop}(\Sigma)$ is stably computable in the queued transmission model.

Theorem 2. *Every predicate on $\text{Pop}(\Sigma)$ stably computable in the two-way model is stably computable in the queued transmission model.*

Proof. Given any protocol in the two-way model, we describe a simulation of it by a protocol in the queued transmission model. Each agent can store up to two states from the two-way protocol. Initially, each agent has one state, which is determined from the input symbol for that agent by the initial state map of the protocol being simulated. Agents transfer states by sending a message. An agent called upon to send will send a message containing the state it has been holding longest. (If it is not currently holding any states, it sends a null message.) Any agent with free space is eligible to receive a message. Whenever an agent receives its second state, it uses the transition function from the two-way protocol to have the two interact, with the state it already has acting as the initiator.

The simulated configuration cannot make any steps that were impossible under the two-way model. Conversely, any sequence of interactions made under the two-way model can be achieved by first having every agent launch all states it holds, and then by repeating the following actions: deliver two states to a particular agent, and then have the agent release both in messages. Therefore, the simulation is faithful. $\square$

8.3 Immediate and Delayed Transmission

Theorem 3. *Every predicate on $\text{Pop}(\Sigma)$ in the class $\mathbf{SLIN} \cap \mathbf{coreMOD}$ is stably computable in the immediate transmission model and also in the delayed transmission model.*

Proof. We first note that both models can stably compute simple threshold predicates and predicates in $\mathbf{MOD}$. This is clear for immediate transmission because the two-way protocols given in Section 8.1 for simple threshold and modulo predicates are immediate transmission protocols. For the delayed transmission model, we specify that a sender sends its state and becomes passive, preserving its previous output. Passive messages are ignored by all receivers. An active receiver combines its data with the data from an active message, remaining active. (The data values u and v are combined as $(u + v) \bmod m$ for a mod predicate or $\min(k, u + v)$ for a simple threshold predicate.) A passive receiver sets its state equal to an active message, becoming active again. In a fair execution eventually every configuration will contain exactly one active agent or active message in transit, and when all agents have received the message, they can compute the correct output value.

Suppose $\psi \in \mathbf{SLIN} \cap \mathbf{coreMOD}$. For each nonempty subset Σ' of the input alphabet Σ , we describe a protocol that computes ψ assuming that Σ' is exactly the set of symbols present in the input x . We can “and” this protocol with one that verifies this assumption (it is a boolean combination of simple threshold predicates, and therefore stably computable in this model), and “or” together the resulting protocols for each choice of Σ' , and the result will stably compute ψ .

Because $\psi \in \mathbf{coreMOD}$, there is a predicate $\psi' \in \mathbf{MOD}$ and an integer $k \geq 0$ such that ψ is k -similar to ψ' with respect to Σ' . Then ψ' is stably computable by an immediate (or delayed) transmission protocol, as is the predicate that is true if x is k -rich with respect to Σ' , so the conjunction of these predicates correctly computes ψ when there are at least k occurrences of each input symbol from Σ' and no occurrences of any other symbol.

In the remaining cases some $\sigma \in \Sigma'$ occurs between 1 and $k - 1$ times. Because $\psi \in \mathbf{SLIN}$, it is stably computable by a two-way protocol. For each input symbol $\sigma \in \Sigma'$ we describe below how to adapt the simulation of the two-way protocol in the proof of Theorem 2 to show that the predicate that is the conjunction of ψ and the predicate that is true when there are between 1 and $k - 1$ occurrences of σ in the input is stably computable in the immediate (or delayed) transmission models. Then we “or” these protocols together for all choices of $\sigma \in \Sigma'$.

The simulation of a two-way protocol by a queued transmission protocol requires queuing in order to prevent agents’ storage from being overwhelmed by messages. However, suppose it may be assumed that at least one and at most $k - 1$ copies of σ can appear in the input. In this simulation, each agent can store up to k states of the original protocol. We give the (at most $k - 1$) agents receiving input σ “tokens” and require that every agent “pay” a token to the recipient every time it transfers a state. The result is that no agent can collect more than k states: one for the state it started with and $k - 1$ transfers. At that point, the agent has all of the tokens and cannot gain any more states. Thus, an agent never runs out of space for the messages it receives.

To see that any step of the original protocol can be simulated in this protocol, consider a simulated configuration with two agents in states q_1 and q_2 . First, the agent holding q_1 sends q_1 in a message. (If the agent does not have enough tokens to do so, deliver a message to it so that it gets the necessary token.) When q_1 is a message in transit, some agent holds no states. Deliver q_1 to such an empty agent. Then, the agent holding q_2 sends q_2 in a message. (Again, deliver a token to this agent if required to make this happen.) Deliver q_2 to the agent that holds q_1 to cause an interaction between q_1 and q_2 . $\square$

Note in particular that immediate and delayed transmission protocols can stably compute the predicate

over the alphabet $\{a, b, c\}$ that is true if there is exactly one c and the number of a 's exceeds the number of b 's. Intuitively, the single c in accepted inputs can be used to implement a kind of flow control on agents' incoming messages. The characterization in Section 14.1 shows that the standard comparison predicate over $\{a, b\}$, which is true if the number of a 's exceeds the number of b 's, is not stably computable by immediate or delayed transmission protocols, essentially because no such control of incoming messages is possible in this case.

8.4 Immediate Observation

Theorem 4. *Every predicate on $\text{Pop}(\Sigma)$ in COUNT_* is stably computable in the immediate observation model.*

Proof. Every predicate in COUNT_* can be expressed as a boolean combination of simple threshold predicates that depend only on the count of one input symbol. The two-way protocol in Section 8.1 for the simple threshold predicate $[x \cdot \sigma \geq k]$ is in fact an immediate observation protocol because only the responder's state is updated in each case. $\square$

The protocol from Section 8.1 uses $k + 1$ states, which can be reduced to k (by removing state 0) if the input alphabet is unary. It can be shown that $k - 1$ states are not sufficient to stably compute this predicate in the immediate observation model in the case of a unary alphabet; the proof is rather involved and will appear elsewhere.

8.5 Delayed Observation

Theorem 5. *Every predicate on $\text{Pop}(\Sigma)$ in COUNT_1 is stably computable in the delayed observation model.*

Proof. The two-way simple threshold protocol in Section 8.1 is also a valid delayed observation protocol for $k = 1$. However, because observation is delayed, an agent may repeatedly advance its state after receiving a message from itself, which means that the protocol fails to compute the correct predicate in the delayed observation model when $k \geq 2$. $\square$

We observe that if agents do not receive messages from themselves, then predicates in COUNT_2 are stably computable in the delayed observation model.

9 Characterization of the Power of the Abstract Model

To complete the characterizations in Figure 2, we have to demonstrate that each model is limited to the indicated power. In Section 14, we consider the one-way models. Here we consider the Abstract Model, which includes the two-way model and the queued transmission model as special cases, and prove the following semilinearity theorem.

Theorem 6. *Every predicate on $\text{Pop}(\Sigma)$ that is stably computable in the Abstract Model is semilinear.*

Because the two-way and queued transmission models are special cases of the Abstract Model, and both models can stably compute all the semilinear predicates over $\text{Pop}(\Sigma)$, we have the following characterizations.

Corollary 7. *The predicates stably computable in (1) the two-way model of population protocols, or (2) the queued transmission model of population protocols are exactly the semilinear predicates on $\text{Pop}(\Sigma)$.*

Stable computation of a predicate by a two-way population protocol with stabilizing inputs was defined in [AAC⁺05]. We do not repeat the definition here, but the idea is that each agent has an input register that may change finitely many times over the course of the execution before stabilizing to its final value, and the goal is to compute a predicate on the multiset of all the agents' final input values. Because this is a more restrictive definition, every predicate stably computable by a two-way protocol with stabilizing inputs is stably computable by a two-way protocol with fixed inputs, but whether the converse held was left as an open problem. However, in [AAC⁺05] it was shown that every semilinear predicate on $\text{Pop}(\Sigma)$ is stably computable by a two-way protocol with stabilizing inputs. Thus we get the following corollary showing that stabilizing inputs do not reduce the power of the standard two-way model.

Corollary 8. *The predicates stably computable by two-way population protocols with stabilizing inputs are exactly the semilinear predicates on $\text{Pop}(\Sigma)$.*

To prove Theorem 6, we must show that the support of any stably-computable predicate is a semilinear set: in particular, that there is a finite set of base points each attached to a finitely-generated cone such that the support is precisely the elements of these cones. The first step, which requires the development of the machinery of Section 10 and is completed in Section 11, is to show that the support can be decomposed into a finite collection of monoid cosets that are *not necessarily finitely generated*. We then proceed, in Sections 12 and 13, to show that any such decomposition can be further decomposed into a finite covering by cosets of finitely generated monoids, which gives us the full result.

10 Groundwork

We assume that ψ is a predicate stably computed by a protocol $(E, \rightarrow, \Sigma, o)$ in the Abstract Model and establish some basic results. These will lead up to the Pumping Lemma of Section 11.

10.1 Monoids, Groups, and Semilinearity

A subset M of $\mathbb{Z}^d$ is a **monoid** if it contains the zero and is closed under addition; if it is also closed under subtraction, M is a **group**. A monoid $M \subseteq \mathbb{Z}^d$ is **finitely generated** if there exists a finite subset $A \subseteq M$ such that every element of M is a sum of elements from A . It is a classic result in abstract algebra that every subgroup of $\mathbb{Z}^d$ is finitely generated [Lan02], but submonoids are not always finitely generated. For example, the following monoid is not finitely generated.

$$M_{\sqrt{2}} = \{(i, j) \in \mathbb{N}^2 : i \leq \sqrt{2}j\}.$$

A subset H of $\mathbb{Z}^d$ is a **group coset** (resp., **monoid coset**) if there exists an element $v \in \mathbb{Z}^d$ such that $H = v + G$ and G is a group (resp., monoid).

Then from the definitions in Section 6.1, we have yet another characterization of linear and semilinear sets. A subset L of $\mathbb{Z}^d$ is **linear** if it is a coset of a finitely generated monoid in $\mathbb{Z}^d$, and is **semilinear** if it is a finite union of linear sets.

10.2 Higman's Lemma

We will make extensive use of some corollaries to Higman's Lemma [Hig52], a fundamental tool in well-quasi-order theory.

Lemma 9. *Every subset of $\mathbb{N}^d$ under the inclusion ordering $\leq$ has finitely many minimal elements.*

Lemma 10. *Every infinite subset of $\mathbb{N}^d$ contains an infinite chain (i.e., an infinite totally ordered sequence).*

These both follow from the fact that Higman's Lemma implies that $\mathbb{N}^d$ is a **well-quasi-order**, that is, a set in which any infinite sequence $a_1, a_2, \dots$ contains elements a_i, a_j with $i < j$ and $a_i \leq a_j$. The special case of Higman's Lemma given in Lemma 9 was proved earlier by Dickson [Dic13].

10.3 Truncation Maps and Their Properties

For each $k \geq 1$, we define a map τ_k from $\mathcal{C}$ to $\mathcal{C}$ by

$$\tau_k(c)(e) := \min(k, c(e)) \text{ for all } e \in E.$$

This map truncates each component of its input to be at most k ; clearly $\tau_k(c) \leq c$ for all $c \in \mathcal{C}$. Two useful properties of τ_k are that it respects both inclusion and addition.

Lemma 11. *For all $c, d \in \mathcal{C}$ and $k \geq 1$, if $c \leq d$ then $\tau_k(c) \leq \tau_k(d)$.*

Proof. For each $e \in E$, we have $c(e) \leq d(e)$, so $\min(k, c(e)) \leq \min(k, d(e))$. Thus $\tau_k(c) \leq \tau_k(d)$. $\square$

Lemma 12. *For all $c, c', d \in \mathcal{C}$ and $k \geq 1$, if $\tau_k(c) = \tau_k(c')$, then $\tau_k(c + d) = \tau_k(c' + d)$.*

Proof. For each $e \in E$, either $c(e) = c'(e)$ or both are at least k . In either case, $\min(k, c(e) + d(e)) = \min(k, c'(e) + d(e))$, so $\tau_k(c + d) = \tau_k(c' + d)$. $\square$

10.4 Truncation and Stability

Truncation is important because membership of a configuration c in the set of output stable configurations $\mathcal{S}$ can be determined from a truncate of fixed size. Let $\mathcal{U} := \mathcal{C} \setminus \mathcal{S}$, the set of **output unstable** configurations.

Lemma 13. *For all $c \leq d$, if $c \in \mathcal{U}$, then $d \in \mathcal{U}$ ($\mathcal{U}$ is closed upward under inclusion).*

Proof. Suppose $c \in \mathcal{U}$ and $c \leq d$. Then either (1) $O(c)$ is undefined, or (2) $O(c) = b$, but for some configuration c' such that $c \xrightarrow{*} c'$ either $O(c')$ is undefined or $O(c') \neq b$. In case (1), $O(d)$ is undefined and $d \in \mathcal{U}$. In case (2), $d = d - c + c \xrightarrow{*} d - c + c'$. If $O(d)$ is not defined, then $d \in \mathcal{U}$. If $O(c')$ is not defined, then $O(d - c + c')$ is not defined, and $d \in \mathcal{U}$. If both $O(d)$ and $O(c')$ are defined, we have that $O(d) = O(c) \neq O(c')$, and $O(d - c + c')$ is either undefined or equal to $O(c')$, so $d \in \mathcal{U}$. Thus $\mathcal{U}$ is closed upwards under inclusion. $\square$

Lemma 14. *There exists $k \geq 1$ such that $c \in \mathcal{U}$ if and only if $\tau_k(c) \in \mathcal{U}$.*

Proof. By Higman's Lemma, only finitely many elements $u_1, \dots, u_n$ are minimal in $\mathcal{U}$, and because $\mathcal{U}$ is upwards closed, $c \in \mathcal{U}$ if and only if $u_i \leq c$ for some i . Let k be the maximum value $u_i(e)$ for all $i \in \{1, \dots, n\}$ and all $e \in E$. Then $\tau_k(u_i) = u_i$ for each i .

Suppose $c \in \mathcal{U}$. Then $u_i \leq c$ for some i , so $u_i = \tau_k(u_i) \leq \tau_k(c)$ by Lemma 11, and thus $\tau_k(c) \in \mathcal{U}$. Conversely, if $\tau_k(c) \in \mathcal{U}$, then $u_i \leq \tau_k(c) \leq c$ for some i , and therefore $c \in \mathcal{U}$. $\square$

Lemma 15. *There exists $k \geq 1$ such that for all $c \in \mathcal{C}$ and $b \in \{0, 1\}$, we have $c \in \mathcal{S}_b$ if and only if $\tau_k(c) \in \mathcal{S}_b$.*

Proof. By Lemma 14, there exists $k \geq 1$ such that for all $c \in \mathcal{C}$, we have $c \in \mathcal{U}$ if and only if $\tau_k(c) \in \mathcal{U}$. Taking the contrapositive, we have $c \in \mathcal{S}$ if and only if $\tau_k(c) \in \mathcal{S}$. Since truncation does not affect output, the conclusion follows. $\square$

10.5 Extensions

We define a map X from $\mathcal{C}$ to subsets of $\mathbb{N}^\Sigma$ as follows.

$$X(c) := \{x \in \mathbb{N}^\Sigma \mid \text{there exists } d \geq c \text{ such that } c + x \xrightarrow{*} d \text{ and } \tau_k(c) = \tau_k(d)\},$$

where k is the constant from the conclusion of Lemma 15. If $c \in \mathcal{S}$, then $X(c)$ is the set of inputs by which c can be pumped. We call such inputs the **extensions** of c . We first prove that pumping does not affect stable output.

Lemma 16. *If $x \in \text{Pop}(\Sigma)$ and $c \in \mathcal{S}$ and $x \xrightarrow{*} c$, then ψ is constant on $x + X(c)$.*

Proof. If $y \in X(c)$, then there exists $d \in \mathcal{C}$ such that $c + y \xrightarrow{*} d$ and $\tau_k(c) = \tau_k(d)$. Since $c \in \mathcal{S}$, by Lemma 15 we have $d \in \mathcal{S}$, and $O(c) = O(d)$. Thus $\psi(x) = \psi(x + y)$. $\square$

We now prove that pumping operations can be composed, i.e., that $X(c)$ is a monoid.

Lemma 17. *$X(c)$ is a monoid for all $c \in \mathcal{C}$.*

Proof. We have $0 \in X(c)$, with $d = c$ as a witness. If $x_1, x_2 \in X(c)$, then there exist d_1, d_2 such that $c \leq d_1$ and $c \leq d_2$ and $\tau_k(c) = \tau_k(d_1) = \tau_k(d_2)$ and $c + x_1 \xrightarrow{*} d_1$ and $c + x_2 \xrightarrow{*} d_2$. Thus

$$c + x_1 + x_2 \xrightarrow{*} d_1 + x_2 = (d_1 - c) + c + x_2 \xrightarrow{*} (d_1 - c) + d_2.$$

Taking $d := d_1 + d_2 - c$, we have $c \leq d$ and $c + x_1 + x_2 \xrightarrow{*} d$ and $\tau_k(c) = \tau_k(d_2) = \tau_k(c + d_2 - c)$ and by Lemma 12, $\tau_k(c + d_2 - c) = \tau_k(d_1 + d_2 - c) = \tau_k(d)$, since $\tau_k(c) = \tau_k(d_1)$. We conclude that $x_1 + x_2 \in X(c)$. $\square$

11 A Pumping Lemma for Stably Computable Predicates

Given a set of inputs $Y \subseteq \text{Pop}(\Sigma)$, a **monoid-coset covering of Y with respect to ψ** is a set $\{(x_i, M_i)\}_{i \in I}$ of pairs of inputs and submonoids of $\mathbb{N}^\Sigma$ such that $Y \subseteq \bigcup_{i \in I} (x_i + M_i)$ and for all $i \in I$, we have $x_i \in Y$ and $\psi(x_i + M_i) = \{\psi(x_i)\}$. We say ψ **admits finite coset coverings** if for all Y there exists a finite monoid-coset covering of Y with respect to ψ . The following lemma states that every stably computable predicate admits a finite monoid-coset covering. We show later (Theorem 21) that any predicate that admits finite coset coverings is semilinear.

Lemma 18. *The (stably computable) predicate ψ admits finite coset coverings.*

Proof. Consider any $Y \subseteq \text{Pop}(\Sigma)$. If Y is a finite set $\{x_1, \dots, x_n\}$ then it has a trivial finite covering $\{(x_i, \emptyset)\}_{1 \leq i \leq n}$. So assume Y is infinite. Let $y_1, y_2, \dots$ be any enumeration of Y such that $y_i \leq y_j$ implies $i \leq j$. (For example, fix an ordering of Σ . Then enumerate the elements of Y ordered by $\leq$, breaking ties according to lexicographic order.) We define a family of sets $B_i \subseteq \text{Pop}(\Sigma) \times \mathcal{S}$ inductively as follows.

$$B_0 := \emptyset.$$

$$B_i := \begin{cases} B_{i-1} & \text{if there exists } (x, c) \in B_{i-1} \text{ such that } y_i \in x + X(c) \\ B_{i-1} \cup \{(y_i, s(y_i))\} \cup \{(y_i, s(c + y_i - x)) \mid (x, c) \in B_{i-1} \text{ and } x \leq y_i\} & \text{otherwise,} \end{cases}$$

where $s(d) \in \mathcal{S}$ is any stable configuration reachable from d . It is easy to see by induction on i that, for all $(x, c) \in B_i$, $x \xrightarrow{*} c$, and in the construction, $s(d)$ is applied only to reachable configurations d , so the existence of $s(d)$ is guaranteed by the requirements of stably computing a predicate. So, Lemma 16 implies that ψ is constant on $x + X(c)$. Let $B := \bigcup_{i \geq 1} B_i$. It follows from 17 that $\{(x, X(c)) \mid (x, c) \in B\}$ is a monoid-coset covering of Y with respect to ψ . We now show that B is finite to prove that this is a finite covering.

Assuming to the contrary that B is infinite, infinitely many different elements of Y appear as first components of elements of B , since each B_i is clearly finite. By Higman's Lemma, there exists an infinite chain $z_1 < z_2 < \dots$ of such elements. Our construction guarantees the existence of associated configurations $\{d_i\}_{i \geq 1}$ such that $(z_i, d_i) \in B$ and $d_i + (z_{i+1} - z_i) \xrightarrow{*} d_{i+1}$ for all $i \geq 1$.

By Higman's Lemma again, there exists an increasing function f such that the sequences $(z_{f(i)})_{i \geq 1}$ and $(d_{f(i)})_{i \geq 1}$ are nondecreasing. Thus the sequence $\tau_k(d_{f(i)})$ reaches a maximum and becomes constant at some index $i = j$. Consequently, we have $z_{f(j+1)} - z_{f(j)} \in X(d_{f(j)})$, which contradicts the membership of $(z_{f(j+1)}, d_{f(j+1)})$ in B . $\square$

Applying this lemma with $Y = \psi^{-1}(1)$ (the support of ψ) we obtain a finite family of monoid cosets $x_i + M_i$ such that

$$\psi^{-1}(1) = \bigcup_i (x_i + M_i).$$

This does not prove semilinearity by itself, since some monoid M_i might not be finitely generated. However, every submonoid of $\mathbb{Z}^1$ is finitely generated, so in the special case of a *unary* alphabet, we have already that ψ is semilinear, and therefore regular.

Corollary 19. *Every stably computable predicate over a unary alphabet is semilinear. Thus, over a unary alphabet, the stably computable predicates are exactly the semilinear (in fact, regular) predicates.*

For example, the unary predicate that is true if the number of input symbols is a power of 2 (or a prime, or any other non-regular predicate) is not stably computable. Another easy corollary suffices to show certain other predicates over non-unary alphabets are not stably computable.

Corollary 20. *Suppose ψ is a stably computable predicate such that $L = \psi^{-1}(1)$ is infinite. Then L contains an infinite linear subset.*

Proof. By Lemma 18 there is a finite monoid-coset covering of L . If L is infinite, some $(x + M) \subseteq L$ in the covering must be infinite. $\square$

As an application, consider the set of inputs over the alphabet $\{a, b\}$ such that the number b 's is the square of the number of a 's. This is an infinite set with no infinite linear subset, and is therefore not stably

computable. Using closure results for stably computable predicates we can then show that the set of all inputs over alphabet $\{a, b, c\}$ such that the number of c 's is the product of the number of a 's and the number of b 's is not stably computable. The existence of a pumping lemma and the negative results for these particular predicates were conjectured in [AAD⁺04, AAD⁺06].

12 Proof of the Semilinearity Theorem: Outline

We are now in a position to give an overview of how we go from the pumping lemma (Lemma 18) of Section 11 to the semilinearity theorem.

Recall the following set of points in $\mathbb{N}^2$.

$$M_{\sqrt{2}} = \{(i, j) : i \leq \sqrt{2}j\}.$$

This is a monoid but is not stably computable. To see this, suppose the contrary. By the Pumping Lemma, there exist monoid cosets $x_i + M_i$ for $1 \leq i \leq m$ such that

$$M_{\sqrt{2}} = \bigcup_{1 \leq i \leq m} (x_i + M_i).$$

Let $v = (-1, \sqrt{2})$; then $x \cdot v > 0$ for all $x \in M_{\sqrt{2}}$. Let $\epsilon = \min_i \{x_i \cdot v\}$. Choose some $y \in M_{\sqrt{2}}$ such that $0 < y \cdot v < \epsilon$. Then for some i , $y \in x_i + M_i$, and $(y - x_i) \cdot v = y \cdot v - x_i \cdot v < \epsilon - \epsilon = 0$. Thus for a sufficiently large $n \in \mathbb{N}$, $(x_i + n(y - x_i)) \cdot v < 0$, which contradicts the fact that $x_i + M_i$ is a subset of $M_{\sqrt{2}}$. The issue here is that the line dividing the positive and negative inputs cannot have an irrational slope if the predicate is stably computable. One ingredient of our proof is a generalization of this idea to separating hyperplanes.

However, to be able to use separating hyperplanes, we first must deal with separating “intermixed” positive and negative inputs using their images in a finite group. For example, consider the following set of points in $\mathbb{N}^2$, which is stably computable.

$$L = \{(i, j) : i < j, (i + j) \text{ is odd}\}.$$

By first separating the points in $\mathbb{N}^2$ by their images $(i \bmod 2, j \bmod 2)$ in the group $\mathbb{Z}_2 \times \mathbb{Z}_2$, we get four subproblems in which lines suffice to separate the positive and negative points.

A further issue is that because of the relationship between the monoids $X(c)$ and their cosets $x + X(c)$, instead of a single separating hyperplane we have to consider finite sets of parallel hyperplanes, which themselves may contain points from the support of ψ . These points can be handled by induction on dimension, but this requires that the main theorem be generalized to consider the intersection of the support of ψ and an arbitrary group coset (where we view each hyperplane as a group coset.) We shall prove the following theorem by induction on the dimension of the group G ; it clearly holds when G has dimension 0, i.e., when G is the trivial group.

Theorem 21. *If ψ admits finite coset coverings then for any group coset $H = x_0 + G \subseteq \mathbb{Z}^\Sigma$, we have $\psi^{-1}(1) \cap H$ is semilinear.*

Note that together with Lemma 18 this implies Theorem 6, as we can take $H = \mathbb{Z}^\Sigma$.

The details of the proof are quite involved and are in Section 13. To summarize briefly, the overall strategy is:

1. By dividing the space into residue classes with respect to appropriately chosen moduli, we can arrange for the vectors from the monoids associated with the cover to appear in all-positive and all-negative regions separated by hyperplanes. In this part of the proof we extend $\mathbb{N}^\Sigma$ to $\mathbb{Z}^\Sigma$ to make use of the fact that all subgroups of $\mathbb{Z}^\Sigma$ are finitely generated. (Section 13.1.)
2. We then further map the problem from $\mathbb{Z}^\Sigma$ to $\mathbb{R}^\Sigma$, and use techniques from convex geometry to show that appropriate hyperplanes separating the monoids indeed exist. (Section 13.2.) We obtain a (looser) separation by **slabs** (the space between two parallel hyperplanes) on the inputs by observing that each input is displaced a uniformly bounded amount from its corresponding extension vector.
3. Moving to $\mathbb{R}^\Sigma$ allows for the possibility of separating hyperplanes with irrational coefficients. Applying the Pumping Lemma as described above, we show that the resulting separating hyperplanes are normal to a vector with rational coordinates and thus correspond to cosets of subgroups of $\mathbb{Z}^\Sigma$. (Section 13.3.)
4. At this stage, we have shown that the positive inputs to the predicate consist of (a) those inputs in the interior of the separated regions, which can be identified first by computing the residue classes of their coordinates and then by identifying which of a finite number of polytopes (given by intersections of half-spaces with rational coordinates) they appear within, and (b) those inputs that lie in some slab. The first class is semilinear as identification of a residue class and identification of membership in a particular polytope are both expressible in Presburger arithmetic. The second class is then shown to be semilinear by induction on dimension, with the base case of dimension 0 being the trivially semilinear case of a single point. (Section 13.4.)

13 Proof of the Semilinearity Theorem: Details

The proof of Theorem 21 is delayed to Section 13.4. First we describe the process of separating inputs in more detail.

13.1 Separating Intermixed Inputs

Let $\psi : \text{Pop}(\Sigma) \rightarrow \{0, 1\}$ be a predicate that admits finite coset coverings. Let G be a subgroup of $\mathbb{Z}^\Sigma$ and let $H = h + G$ be a coset of G . Take $\{(a_i, M_i)\}_{i \in I}$ and $\{(b_j, N_j)\}_{j \in J}$ to be finite monoid-coset covers of $\psi^{-1}(0) \cap H$ and $\psi^{-1}(1) \cap H$ respectively. Assume without loss of generality that $M_i, N_j \subseteq G$ by intersecting with G if necessary. Define $K(i, j) := \mathbb{Z}(M_i \cap N_j)$, the group generated by the intersection of M_i and N_j .

Lemma 22. *For all $i \in I$ and $j \in J$, no coset of $K(i, j)$ intersects both $a_i + M_i$ and $b_j + N_j$.*

Proof. If we suppose the lemma is false, then there exist $x \in a_i + M_i$ and $x' \in b_j + N_j$ such that $(x - x') \in K(i, j)$. Because $M_i \cap N_j$ is a monoid, we can rewrite $x - x'$ as a difference $y' - y$ where $y \in M_i$ and $y' \in N_j$. Thus $x + y = x' + y'$. The former is in $a_i + M_i$, whereas the latter is in $b_j + N_j$. This contradicts the fact that $a_i + M_i$ and $b_j + N_j$ are disjoint (they have different predicate values). $\square$

Define

$$K := \bigcap_{K(i, j) \text{ has finitely many cosets in } G} K(i, j).$$

In addition to having finitely many cosets in G , the group K inherits the relevant properties of the groups $K(i, j)$ included in its defining intersection.

Lemma 23. *For all $i \in I$ and $j \in J$ such that $K(i, j)$ has finitely many cosets in G , no coset of K intersects both $a_i + M_i$ and $b_j + N_j$.*

Proof. Since $K(i, j) \supseteq K$, each coset of K is contained in a coset of $K(i, j)$. The result follows from Lemma 22. $\square$

Since all group cosets are semilinear, we can use membership in the cosets of K to separate those pairs of monoid cosets that are intermixed. For the others, we have to take another approach.

13.2 Separating with Hyperplanes

Let $g_1 + K, \dots, g_n + K \subseteq G$ be the cosets of K in G . For each $\ell \in \{1, \dots, n\}$ define $H_\ell := h + g_\ell + K$. Note that $\cup_\ell H_\ell = H$. If two monoid cosets $a_i + M_i$ and $b_j + N_j$ meet both meet some coset $h + (g_\ell + K) \subseteq H$, we show it is possible to separate $H_\ell \cap (a_i + M_i)$ from $H_\ell \cap (b_j + N_j)$ with a hyperplane (except for a set of lower dimension). Define $I_\ell := \{i \in I \mid (a_i + M_i) \cap H_\ell \neq \emptyset\}$ and $J_\ell := \{j \in J \mid (b_j + N_j) \cap H_\ell \neq \emptyset\}$. If $i \in I_\ell$ and $j \in J_\ell$, then H_ℓ , which is a coset of K , intersects both $a_i + M_i$ and $b_j + N_j$, so $K(i, j)$ has infinitely many cosets in G , by Lemma 23.

At this point we turn to the methods of geometry. We pass from groups to vector spaces by working in the vector space closure $\mathbb{R}G$ of G in $\mathbb{R}^\Sigma$. Instead of monoids, we consider the convex cones they generate: the set of nonnegative linear combinations of monoid elements. We connect the geometry to the algebra by observing that $K(i, j)$ has finitely many cosets in G if and only if $\mathbb{R}K(i, j)$, the vector space closure of $K(i, j)$, is all of $\mathbb{R}G$. Thus if two monoid cosets are not intermixed, their intersection of their associated monoids has strictly smaller dimension than G . This allows us to separate these monoids with a hyperplane.

Formally, given sets of vectors U and U' , a set of nonzero vectors V **distinguishes** U and U' if for all $u \in U$ and $u' \in U'$, there exists $v \in V$ such that $(u \cdot v)(u' \cdot v) \leq 0$; that is, either one dot product is zero or one is negative and the other is positive. Define $\widehat{M}_\ell := \bigcup_{i \in I_\ell} M_i$ and $\widehat{N}_\ell := \bigcup_{j \in J_\ell} N_j$. The goal of this subsection is to show the existence of a finite set of vectors V that distinguishes $\widehat{M}_\ell$ from $\widehat{N}_\ell$. The main tool we use is the Separating Hyperplane Theorem from convex geometry. Note that $\text{int } U$ denotes the interior of U .

Theorem 24. (*Separating Hyperplane Theorem [Lay92]*) *If U and U' are convex subsets of $\mathbb{R}^d$ with nonempty interiors such that $\text{int } U \cap \text{int } U' = \emptyset$, then there exists $v \in \mathbb{R}^d$ such that $u \cdot v \leq 0$ for all $u \in U$ and $u' \cdot v \geq 0$ for all $u' \in U'$.*

Unfortunately, M_i and N_j are not convex. Thus we are forced to consider the convex cones that they generate. We need Carathéodory's theorem, another result from convex geometry, to verify the seemingly trivial fact that the intersection of these cones lies in a proper vector subspace of $\mathbb{R}G$.

Theorem 25. (*Carathéodory's Theorem [Lay92]*) *For any set of vectors $Y \subseteq \mathbb{R}^d$, if $x \in \mathbb{R}_+ Y$, then there exists a linearly independent subset $Y' \subseteq Y$ such that $x \in \mathbb{R}_+ Y'$.*

Lemma 26. *For all $i \in I_\ell$ and $j \in J_\ell$, the set $Z = \mathbb{R}_+ M_i \cap \mathbb{R}_+ N_j$ is contained in a proper vector subspace of $\mathbb{R}G$.*

Proof. Suppose to the contrary. Then the vector subspace $\mathbb{R}Z$ is all of $\mathbb{R}G$ and has a basis $\{z_1, \dots, z_d\} \subseteq Z$. By perturbing this basis to have rational coordinates, we can assume without loss of generality that $z_s \in \mathbb{Q}G$ as well for all $s \in \{1, \dots, d\}$.

By Theorem 25, for each s , there is a linearly independent set $Y_s \subseteq M_i$ such that $z_s \in \mathbb{R}_+ Y_s$. When we write each z_s as its unique linear combination of elements in Y_s , we see by linear algebra over $\mathbb{Q}G$ that in fact $z_s \in \mathbb{Q}_+ M_i$. Repeating this argument with N_j yields that $z_s \in \mathbb{Q}_+ M_i \cap \mathbb{Q}_+ N_j$. We clear denominators to find numbers m_s such that $m_s z_s \in M_i \cap N_j$. The set $\{m_s z_s\}$, however, is a basis for $\mathbb{R}G$, which contradicts the fact that $\mathbb{R}K(i, j)$ is not all of $\mathbb{R}G$. $\square$

Lemma 27. *For all $i \in I_\ell$ and $j \in J_\ell$, there exists a nonzero vector v such that $\{v\}$ distinguishes I_ℓ and J_ℓ .*

Proof. If either M_i or N_j is contained in a proper vector subspace of $\mathbb{R}G$, then take v to be normal to that subspace.

Otherwise, consider the sets $U := \mathbb{R}_+ M_i$ and $U' := \mathbb{R}_+ N_j$. The intersection of their interiors is both open and contained in a proper vector subspace of $\mathbb{R}G$. Thus U and U' have no interior point in common. Therefore, by Theorem 24 there exists $v \in \mathbb{R}G$ such that for all $u \in U$, we have $u \cdot v \leq 0$; and for all $u' \in U'$, we have $u' \cdot v \geq 0$. It follows that $\{v\}$ distinguishes M_i and N_j . $\square$

To obtain a set of vectors that distinguish $\widehat{M}_\ell$ and $\widehat{N}_\ell$, we put together all of the individual distinguishing vectors.

Lemma 28. *For all $1 \leq \ell \leq n$, there exists a finite set of vectors V that distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$.*

Proof. The set $V := \{v(i, j) \mid i \in I_\ell \text{ and } j \in J_\ell\}$ distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$, where $v(i, j)$ is the vector from Lemma 27 such that $\{v(i, j)\}$ distinguishes M_i and N_j . $\square$

Combining results in this section and the previous one, we have some powerful criteria for determining the predicate value of an input.

Lemma 29. *Let V be a set of vectors that distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$. Suppose $x \in \widehat{M}_\ell$ and $y \in H_\ell$ are such that for all $j \in J_\ell$ and $v \in V$, we have $(x \cdot v)((y - b_i) \cdot v) > 0$. Then $\psi(y) = 0$.*

Proof. Suppose to the contrary that $\psi(y) = 1$. Then there exists $j \in J_\ell$ such that $(y - b_j) \in N_j$. The set V , however, fails to distinguish x and $y - b_j$, which is a contradiction. $\square$

Clearly, this lemma has an analogous counterpart that establishes sufficient conditions for $\psi(y) = 1$.

13.3 Achieving Rationality

The chief obstacle yet to be overcome is that a distinguishing set of vectors might include vectors with irrational coordinates. In order to rule out predicates like $\psi(r, s) := [r < (\sqrt{2})s]$, we need to show that we can always distinguish $\widehat{M}_\ell$ and $\widehat{N}_\ell$ by vectors with integral coordinates. We must use for a second time the fact that ψ admits finite coset covers.

Lemma 30. *For all ℓ there exists a finite set of vectors from $\mathbb{Z}^\Sigma$ that distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$.*

Proof. Since we can clear denominators, it is enough to find a set of vectors in $\mathbb{Q}^\Sigma$ that distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$. By Lemma 28 there exists a finite set of vectors V that distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$. Assume that $V \setminus \mathbb{Q}^\Sigma$ has minimum cardinality, that is, as few vectors in V have irrational components as possible. To

avoid a special case later, we assume without loss of generality that V contains the standard basis Σ of $\mathbb{R}^\Sigma$. We show that this set V contains only vectors in $\mathbb{Q}^\Sigma$.

Suppose to the contrary that there exists $v \in V \setminus \mathbb{Q}^\Sigma$. By assumption, the set $V' := V \setminus \{v\}$ cannot distinguish $\widehat{M}_\ell$ and $\widehat{N}_\ell$. Thus there exist $i \in I_\ell$ and $j \in J_\ell$ and $x \in M_i$ and $y \in N_j$ such that for all $v' \in V'$ we have $(x \cdot v')(y \cdot v') > 0$.

We consider two cases. In the first case, for all choices of x and y we have $(x \cdot v)(y \cdot v) = 0$. Then at least one vector in each problem pair is normal to v . By linear algebra over $\mathbb{Q}^\Sigma$, there exists a vector $v' \in \mathbb{Q}^\Sigma$ such that if $w \in G$ is normal to v , then w is normal to v' . Thus $V' \cup \{v'\}$ distinguishes $\widehat{M}_\ell$ and $\widehat{N}_\ell$, which is a contradiction, since $(V' \cup \{v'\}) \setminus \mathbb{Q}^\Sigma$ has fewer elements than $V \setminus \mathbb{Q}^\Sigma$.

In the second case, we have x and y such that $(x \cdot v')(y \cdot v') > 0$ for all $v' \in V'$ and $(x \cdot v)(y \cdot v) < 0$. Assume without loss of generality that $x \cdot v < 0 < y \cdot v$ by taking $-v$ instead of v if necessary. Let $\Omega := \{w \in \mathbb{R}G \mid v' \cdot w > 0 \text{ for all } v' \in V' \text{ and } w(q) > 0 \text{ for all } q \in \Sigma\}$. Clearly Ω is an open set. Also, $y \in \Omega$, since by the assumption that $\Sigma \subseteq V$ we have $y(q) > 0$ for all $q \in \Sigma$. Given that Ω is a nonempty open set, we can extend x, y to a basis $w_1 = x, w_2 = y, \dots, w_m$ such that $w_s \in \Omega$ for all $s \geq 2$. By perturbing each w_s slightly to have rational coordinates and clearing denominators, we assume without loss of generality that $w_s \in \Omega \cap K$. There exists s such that $(w_s \cdot v)/(w_1 \cdot v)$ is irrational, since otherwise some scalar multiple of v belongs to $\mathbb{Q}^\Sigma$.

In consequence $\mathbb{N}(x \cdot v) + \mathbb{N}(w_s \cdot v)$ is dense in $\mathbb{R}$, so we can find sequences of positive integers m_t and m'_t such that

$$m_t(x \cdot v) + m'_t(w_s \cdot v)$$

is a negative monotone increasing sequence of real numbers approaching 0 as t approaches infinity. For all $v' \in V$, the points $(m_t x + m'_t w_s)_{t \geq 1}$ lie on the same side of the hyperplane normal to v , and as a sequence they approach the hyperplane normal to v arbitrarily closely. By another application of Higman's Lemma, there is an increasing function f such that $(m_{f(t)}, m'_{f(t)})$ is an increasing sequence.

Let $z \in (a_i + M_i) \cap H_\ell$. There exists some constant $c \geq 0$ such that for each r , the points $((z + (c + m_{f(t)}x + m'_{f(t)}w_s) - b_r)_{t \geq 1})$ all lie on the same side of each hyperplane as x . It is easily verified that each of these points belongs to H_ℓ . Thus by Lemma 29, $\psi(z + (c + m_{f(t)}x + m'_{f(t)}w_s))$ is constantly 0. Apply the pumping lemma (Lemma 18) again to these inputs to obtain a finite cover. Then there exist $t_1 < t_2$ such that $(m_{t_2} - m_{t_1})x + (m'_{t_2} - m'_{t_1})w_s \in P$, where the monoid coset $(z + m_{t_1}x + m'_{t_1}w_s) + P$ belongs to the cover. Pumping $z + (c + m_{t_1})x + m'_{t_1}w_s$ by a sufficiently large multiple of $(m_{t_2} - m_{t_1})x + (m'_{t_2} - m'_{t_1})w_s$ yields an element z' such that $\psi(z') = 0$, but for each r , we have that $z' - a_r$ is on the same side of each hyperplane as w_s , which contradicts Lemma 29. $\square$

13.4 Proof of Theorem 21

We can now prove Theorem 21 by induction on the dimension of G (the cardinality of the largest linearly independent subset of G .)

Proof. If the dimension of G is zero, then H is a single point and the result holds. If the dimension of G is greater than zero, then by Lemmas 23 and 30, there exist a group K and finite distinguishers $V_\ell \subseteq \mathbb{Z}^\Sigma$ for all cosets H_ℓ of K in H .

For each distinguishing vector $v \in V_\ell$, consider the set of points $x \in H_\ell$ such that it is not the case that the following numbers are either all negative or all positive: $(x - a_i) \cdot v$ for $i \in I_\ell$ and $(x - b_j) \cdot v$ for $j \in J_\ell$. This set has finite width in the direction of v . Thus it can be written as the union of finitely many

cosets of a group of smaller dimension. The key to the induction is that any point not in the union of these sets over the different x satisfies the hypotheses of one of the variants of Lemma 29.

Define the sets

$$B := \bigcup_{\ell} \{x \in H_{\ell} \mid \exists y \in \widehat{N}_{\ell} \text{ such that } ((x - b_j) \cdot v)(y \cdot v) > 0 \text{ for all } j \in J_{\ell} \text{ and } v \in V_{\ell}\}$$

$$B' := \bigcup_{\ell} \{x \in H_{\ell} \mid \exists y \in \widehat{M}_{\ell} \text{ such that } ((x - a_i) \cdot v)(y \cdot v) > 0 \text{ for all } i \in I_{\ell} \text{ and } v \in V_{\ell}\}.$$

By Lemma 29 we have $B \subseteq \psi^{-1}(1) \subseteq B'$. It is not difficult to verify that B and B' are semilinear. Moreover, $B' \setminus B$ is a union of finitely many group cosets of dimension less than G . It follows by inductive hypothesis that $\psi^{-1}(1) \cap (B' \setminus B)$ is semilinear, and thus that ψ itself is semilinear. $\square$

14 Characterizations of the One-Way Models

In this section, we complete the characterizations of the one-way models in Figure 2 by showing that they are limited to the indicated power.

14.1 Immediate and Delayed Transmission

We first consider the immediate and delayed transmission models to prove the following converse to Theorem 3.

Theorem 31. *Let ψ be a predicate that is stably computable by an immediate or delayed transmission protocol. Then ψ is in $\mathbf{SLIN} \cap \mathbf{coreMOD}$.*

Thus, for example, the comparison predicate, true if there are more a 's than b 's in the input, is not stably computable in the immediate or delayed transmission model, because it is not in $\mathbf{coreMOD}$. Intuitively, the weakness of delayed transmission compared with queued transmission is in the inability of a receiver to refuse messages temporarily; in effect, by delivering a number of copies of some message to a receiver and and forcing it back into the same state, we cause the whole collection of messages to “disappear” from the computation.

Because a delayed transmission protocol is a special case of a queued transmission protocol and an immediate transmission protocol is a special case of a two-way protocol, Corollary 7 shows that ψ is in $\mathbf{SLIN}$. To see that ψ is also in $\mathbf{coreMOD}$ requires some preliminary lemmas. Consider any delayed or immediate transmission protocol that stably computes ψ , and consider any nonempty subset Σ' of Σ .

To unify the treatment of the two kinds of protocols, we assume that in a delayed transmission protocol, the sender simply sends its current state; thus, we need not distinguish between states and messages in our description. Formally, in this case the set of messages consists of a disjoint copy of the set of states, distinguished by the phrases “the state q_i ” and “the message q_i .” In the case of immediate transmission, an interaction $(p, q) \mapsto (\delta_1(p), \delta_2(p, q))$ is described as the sender sending message p and going to state $\delta_1(p)$, and the receiver receiving message p and going to state $\delta_2(p, q)$.

We let Q' denote the set of states that appear in configurations reachable from inputs that contain only symbols from Σ' . For states $q, q' \in Q'$ of the protocol, we say that q **can reach** q' if there is some finite sequence of messages (from Q') sent and received by an agent in state q that enable it to enter state q' . Formally, q can reach q' if there is a sequence of states $q = q_1, q_2, \dots, q_k = q'$ such that, for $2 \leq i \leq k$, either $q_i = \delta_s(q_{i-1})$ or $q_i = \delta_r(p, q_{i-1})$ for some message $p \in Q'$.

The following lemma shows that, for any message p , a sufficient number of copies of p can be “absorbed” by an agent in some state q , returning that agent to state q .

Lemma 32. Fix any message $p \in Q'$. Define $f_p(q) = \delta_r(p, q)$. There exists a state $q \in Q'$ and a positive integer n such that $f_p^{(n)}(q) = q$.

Proof. For any state $r \in Q'$, the sequence of states $r, f_p(r), f_p^{(2)}(r), \dots$ in Q' must be eventually periodic of some period n . Choosing q to be any state in the periodic part, we see that $f_p^{(n)}(q) = q$. $\square$

If n is a positive integer and $q_1, q_2 \in Q'$ are states, we say that q_1 and q_2 are n -**substitutable** if there exist an input x containing only symbols from Σ' and a configuration b of states, such that the configurations $c = nq_1 + b$ and $d = nq_2 + b$ are reachable from $I(x)$. We note that if q_1 and q_2 are n -substitutable, then they are mn -substitutable for any positive integer m . We define two states $q_1, q_2 \in Q'$ to be **substitutable** if there is some positive integer n such that they are n -substitutable. Substitutability is reflexive and symmetric by definition; we now show it is transitive.

Lemma 33. Let $q_1, q_2, q_3 \in Q'$ be states and assume that q_1 and q_2 are substitutable and q_2 and q_3 are substitutable. Then q_1 and q_3 are substitutable.

Proof. For some input x containing only symbols from Σ' , configuration b of states and integer n , the configurations $c = nq_1 + b$ and $d = nq_2 + b$ are reachable from $I(x)$. Similarly, for some input y containing only symbols from Σ' , configuration b' of states and positive integer m , the configurations $c' = mq_2 + b'$ and $d' = mq_3 + b'$ are reachable from $I(y)$.

Consider the input $z = mx + ny$, which contains only input symbols from Σ' . From $I(z)$ there is a computation reaching $mc + nc'$, which is equal to $mnq_1 + (mnq_2 + mb + nb')$, and also a computation reaching $md + nd'$, which is equal to $mnq_3 + (mnq_2 + mb + nb')$. Thus q_1 and q_3 are mn -substitutable. $\square$

Now we show that reachability implies substitutability.

Lemma 34. If $q_1, q_2 \in Q'$ are states such that q_1 can reach q_2 then q_1 and q_2 are substitutable.

Proof. Because substitutability is transitive, it suffices to consider the case in which q_2 is reachable in one step from q_1 . For every state $q \in Q'$, we define c_q to be any configuration of states containing q that is reachable from an input containing only symbols from Σ' . We consider two cases.

If q_1 reaches q_2 by a send step, then by Lemma 32 we may choose a state $q_3 \in Q'$ and a positive integer n such that $f_{q_1}^{(n)}(q_3) = q_3$. (Intuitively, this means an agent in state q_3 will be back in q_3 after receiving n copies of message q_1 .) We choose $b = n(c_{q_1} - q_1) + c_{q_3}$, and let $c = nq_1 + b$, so that $c = nc_{q_1} + c_{q_3}$. Then c is reachable from $I(x)$ for some input x containing only symbols from Σ' .

And from c we may have each of n agents in state q_1 transmit a message (and reach state q_2) to a single agent in state q_3 , which will again be in state q_3 after receiving the n copies of q_1 . Formally, it is easy to see by induction on j that $c \xrightarrow{*} (n - j)q_1 + jq_2 + b - q_3 + f_{q_1}^{(j)}(q_3)$ for $0 \leq j \leq n$. Taking $j = n$, we have that $d = nq_2 + b$ is reachable from c , and therefore from $I(x)$, so q_1 and q_2 are substitutable.

Suppose q_1 reaches q_2 by a receive step, say of message $q_3 \in Q'$. Let q_4 be the state reached from state q_3 after a send. Then by Lemma 32, there is a state $q_5 \in Q'$ and a positive integer n such that $f_{q_3}^{(n)}(q_5) = q_5$. (This means that after receiving n copies of message q_3 an agent that starts in state q_5 will be back in state q_5 .) We choose

$$b = n(c_{q_1} - q_1) + n(c_{q_3} - q_3) + nq_4 + c_{q_5},$$

and and let

$$c = nq_1 + b = nc_{q_1} + n(c_{q_3} - q_3) + nq_4 + c_{q_5}.$$

Now $c' = nc_{q_1} + nc_{q_3} + c_{q_5}$ is reachable from $I(x)$ for some input x containing only symbols from Σ' . It is easy to see by induction on j that, for $0 \leq j \leq n$, $c' \xrightarrow{*} nc_{q_1} + nc_{q_3} - jq_3 + jq_4 + c_{q_5} - q_5 + f_{q_3}^{(j)}(q_5)$ by a sequence of steps in which j agents in state q_3 send a message (and go to state q_4) to a single agent that started in state q_5 . Taking $j = n$, we see that $c' \xrightarrow{*} nc_{q_1} + nc_{q_3} - nq_3 + nq_4 + c_{q_5} = c$. Also, $d = nq_2 + b = n(c_{q_1} - q_1) + nq_2 + n(c_{q_3} - q_3) + nq_4 + c_{q_5}$ is reachable from c' by a computation in which we pair up n agents in state q_3 with n agents in state q_1 and in each of the n pairs have the agent in state q_3 send a message to the agent in state q_1 , transforming the sender into state q_4 and the receiver into state q_2 . Thus, in this case also, q_1 and q_2 are substitutable. $\square$

Thus we have the following pumping lemma for predicates stably computable by immediate and delayed transmission protocols.

Lemma 35. *For every input symbol $\sigma \in \Sigma'$ there exist positive integers k and n such that for every input $y \in \text{Pop}(\Sigma)$ that is k -rich with respect to Σ' , y and $y + n\sigma$ are both accepted or both rejected.*

Proof. Let k_0 be the constant in Lemma 15 such that k_0 -truncates are sufficient to determine membership in the set $\mathcal{S}$ of output stable configurations for the protocol. We choose k_1 to be $|Q'|(k_0 - 1) + 1$.

Let σ be any input symbol from Σ' and let $q = \iota(\sigma)$, the initial state associated with σ . Let $R \subseteq Q'$ be the set of states that q can reach, and consider any $q' \in R$. By Lemma 34, q and q' are $n_{q'}$ -substitutable for some positive integer $n_{q'}$. Let n be the least common multiple of the $n_{q'}$ for all $q' \in R$; q and q' are n -substitutable. That is, there exists an input $x_{q'}$ containing only symbols from Σ' and a configuration of states $b_{q'}$ such that the configurations $c_{q'} = nq + b_{q'}$ and $d_{q'} = nq' + b_{q'}$ are reachable from $I(x_{q'})$.

Let x be the sum of all $x_{q'}$ over $q' \in R$. We choose k_2 to be the maximum multiplicity of any input symbol in x . Let $k = \max(k_1, k_2)$ and let y be any input that is k -rich with respect to Σ' . Then $y \geq x$ and y contains only symbols from Σ' .

Consider a computation from $I(y)$ that first takes $I(x)$ to the sum $s = \sum_{q' \in R} c_{q'}$ and then proceeds to an output stable configuration e . Since y is k -rich with respect to Σ' , there are at least $k \geq k_1$ agents. By the pigeonhole principle, some state q' appears with multiplicity at least k_0 in e . Note $q' \in R$.

Now from $I(y + n\sigma) = I(y) + nq$ we consider a computation that takes $I(x - x_{q'})$ to $s - c_{q'}$ and $I(x_{q'})$ to $d_{q'}$, giving

$$I(y - x) + nq + (s - (nq + b_{q'})) + nq' + b_{q'} = I(y - x) + s + nq'.$$

Now we continue by running $I(y - x) + s$ to e , giving $e + nq'$, which is output stable because $\tau_{k_0}(e) = \tau_{k_0}(e + nq')$. Thus y and $y + n\sigma$ are both accepted or both rejected. $\square$

Now we can conclude the proof of Theorem 31.

Proof. For each alphabet symbol $\sigma \in \Sigma'$, by Lemma 35 there exist integers k_σ and n_σ such that for all inputs y that are k_σ -rich with respect to Σ' , $y + n_\sigma\sigma$ is accepted if and only if y is accepted. Let k be the maximum of the k_σ 's over $\sigma \in \Sigma'$. Then the acceptance or rejection of an input y that is k -rich with respect to Σ' depends only on the values of the number of occurrences of σ modulo n_σ , which implies that ψ is k -similar with respect to Σ' to a predicate in **MOD**. Because Σ' was an arbitrary nonempty subset of Σ , this shows that $\psi \in \text{coreMOD}$. $\square$

14.2 Immediate Observation

In the immediate observation model, transitions are of the form $(p, q) \mapsto (p, q')$ and there is no multiset of undelivered messages. We have shown in Theorem 4 that for any constant k , an immediate observation protocol can count the number of copies of each input symbol up to k . However, this is also the extent of its power.

Theorem 36. *Let ψ be a predicate that is stably computable by an immediate observation protocol. Then $\psi \in \mathbf{COUNT}_*$.*

Proof. Let k be the constant in Lemma 15 such that k -truncates are sufficient to determine membership in the set $\mathcal{S}$ of output stable configurations. Assume that x is an input such that some input symbol σ occurs with multiplicity at least $k' = |Q|(k - 1) + 1$ in x . Let $c_0, c_1, c_2, \dots, c_m$ be an execution from the initial configuration $c_0 = I(x)$ to an output stable configuration c_m . Then, $c_i = c_{i-1} - p_i + p'_i$ for some $p_i, p'_i \in Q$. We define $s_i(j)$ inductively to represent the state of the j th agent with input σ in c_i , as follows. Let $s_0(j) = \iota(\sigma)$. If the multiset $\{s_{i-1}(j) : 1 \leq j \leq k'\}$ is contained in c_i , then $s_i = s_{i-1}$. Otherwise, there

is some $\hat{j}$ such that $s_{i-1}(\hat{j}) = p_i$, and we define $s_i(j) = \begin{cases} p'_i & \text{if } j = \hat{j} \\ s_{i-1}(j) & \text{otherwise.} \end{cases}$

By the pigeonhole principle, the multiset $\{s_m(j) \mid 1 \leq j \leq k'\}$ contains some state q' with multiplicity at least k . Choose j' such that $s_m(j') = q'$. We introduce a “clone” of this agent into the execution. Formally, we show by induction that $I(x + \sigma) \xrightarrow{*} c_i + s_i(j')$. When $i = 0$, $I(x + \sigma) = c_0 + s_0(j')$. Assume $I(x + \sigma) \xrightarrow{*} c_{i-1} + s_{i-1}(j')$. If $q_i(j') = s_{i-1}(j')$, then the claim is clearly true. Otherwise, $c_i + s_i(j') = c_{i-1} - p_i + 2p'_i$, which is reachable from $c_{i-1} + s_{i-1}(j') = c_{i-1} + p_i$ by having two agents in state p_i change to state p'_i by observing some state in $c_{i-1} - p_i$.

Thus, $I(x + \sigma) \xrightarrow{*} c_m + q'$. Because the multiplicity of q' in c_m is at least k , $\tau_k(c_m) = \tau_k(c_m + q')$. Therefore, since c_m is output stable, so is $c_m + q'$. Moreover, $c_m + q'$ has the same output as c_m , so $\psi(x) = \psi(x + \sigma)$. Because this holds of any input symbol σ of multiplicity at least k' in x , we have $\psi \in \mathbf{COUNT}_{k'} \subset \mathbf{COUNT}_*$. $\square$

We may also consider a variant of the immediate observation model in which an agent may interact with itself, that is, a rule $(p, p) \mapsto (p, q)$ can be applied to a single agent in state p to change it to state q . This is the model **with mirrors**. The two versions of the model are equal in power, as they can simulate each other, using bit-flipping protocols to avoid or permit self-interactions.

Theorem 37. *The same predicates are stably computable by immediate observation protocols in the models with and without mirrors.*

Proof. We may assume that n , the population size, is at least 3, since any predicate on 3 or fewer agents is computable in either model; this case can be detected and handled separately in either kind of protocol.

Given an immediate observation protocol that stably computes a predicate ψ in the model without mirrors, we can derive another protocol that stably computes ψ in the model with mirrors. For every old state q we introduce a new state q' and for every old transition $(p, q) \mapsto (p, r)$, where $p \neq q$, we introduce variants of the transition with all combinations of primed and unprimed states p, q , and r . We replace every old transition $(p, p) \mapsto (p, q)$ with four new transitions: $(p, p) \mapsto (p, p')$ and $(p', p') \mapsto (p', p)$, as well as $(p, p') \mapsto (p, q)$ and $(p', p) \mapsto (p', q)$. In the mirrored model, this allows an agent to flip back and forth between p and p' by itself, but to leave these two states, it must interact with someone other than itself (witnessed by having different “primation”).

For the other direction, given an immediate observation protocol in the model with mirrors that stably computes a predicate ψ , we can again introduce primed and unprimed versions of each state. For each ordered pair of states p and q , we have rules $(p, q) \mapsto (p, q')$ and $(p, q') \mapsto (p, q)$, which allow a state to flip between primed and unprimed as long as there is at least one unprimed state. Because the input states are unprimed, these rules cannot eliminate the last unprimed state. In addition, for every rule $(p, q) \mapsto (p, r)$ with $p \neq q$ in the protocol with mirrors, there is a rule $(p', q) \mapsto (p', r)$ in the protocol without mirrors; this allows steps involving two agents in different states to be simulated by priming the initiator, unpriming the responder, and taking the step. For every rule $(p, p) \mapsto (p, r)$ in the protocol with mirrors, there are rules $(q', p') \mapsto (q', r')$ for every original state q . If a step involves two agents in state p , it can be simulated by priming both agents and taking the step. If a step involves one agent in state p , it can be simulated by priming that agent (to get p') and any other agent (to get q') and taking the step. Note that this simulation can be carried out if there are at least three agents in the population because there is at least one unprimed agent in every configuration reachable from an input configuration. $\square$

14.3 Delayed Observation

Here, we prove the converse of Theorem 5, showing that the delayed observation model is the weakest of the one-way models we defined.

Theorem 38. *Suppose ψ is stably computed by a delayed observation protocol. Then ψ is in $\mathbf{COUNT}_1$.*

Proof. We show that for any input x , if $\sigma \in x$ then $\psi(x + \sigma) = \psi(x)$, which implies that ψ is completely determined by the presence or absence of each input symbol and hence is in $\mathbf{COUNT}_1$.

Consider the finite graph whose nodes are configurations reachable from $I(x)$ that contain no messages in transit, with a directed edge from c to c' if $c \xrightarrow{*} c'$. A **final** strongly connected component of this graph is one from which no other strongly connected component of the graph is reachable. From $I(x)$ we can reach a configuration in a final strongly connected component $\mathcal{F}$ of this graph. Let $\hat{\mathcal{F}}$ denote all the configurations d , including those with undelivered messages, such that $c \xrightarrow{*} d$ for some $c \in \mathcal{F}$. For any configurations d and d' in $\hat{\mathcal{F}}$, $d \xrightarrow{*} d'$ by first delivering all messages in d . This implies that all configurations in $\hat{\mathcal{F}}$ are output stable.

The set T of states that occur in configurations in $\hat{\mathcal{F}}$ is closed, that is, if $p, q \in T$ and $(p, q) \mapsto (p, q')$, then $q' \in T$. To see this, assume not. Then, take a configuration c in $\hat{\mathcal{F}}$ that contains p and let an agent in state p send a message, putting p into messages in transit. Now mimic a computation from d to a configuration d' in $\hat{\mathcal{F}}$ containing q , leaving the message p undelivered. Then deliver p to an agent in state q , arriving at a configuration in $\hat{\mathcal{F}}$ containing q' , a contradiction.

Now consider any σ in x . Consider an execution that begins in $I(x)$ and ends in an output stable configuration c in $\mathcal{F}$. There must be a sequence of states $q_0, q_1, \dots, q_m$ where $q_0 = \iota(\sigma)$, q_m appears in c , and $(p_i, q_{i-1}) \mapsto (p_i, q_i)$ for some state p_i that appeared in some configuration during the execution. Starting from the configuration $I(x + \sigma) = I(x) + q_0$, we can reach a configuration $c + q_0$ that also has one additional copy of each message $p_1, \dots, p_m$ left in transit. By delivering all of the m messages to the agent in state q_0 in order, we reach the configuration $c + q_m$. Because T is closed and the states of $c + q_m$ are all in T , $c + q_m$ is output stable and therefore $\psi(x + \sigma) = \psi(x)$. $\square$

If we assume that no agent ever receives its own message, then the power of the model increases only slightly: it becomes possible for an agent in a unique state to observe that it never encounters a twin, and the stably computable predicates are $\mathbf{COUNT}_2$ in this case. The proof is a straightforward extension of the proof of Theorem 38.

15 Local Fairness Is Weak Even with Unbounded States

In this section, we consider a strongly anonymous message-passing model with the following local fairness condition: if some agent sends a particular message m infinitely often, then each agent receives message m infinitely often. This model turns out to be surprisingly weak. Even if the states of processes and the lengths of messages may grow without bound, protocols in this model cannot distinguish two multisets of inputs if the same set of values appears in each. Since this model subsumes the one-way finite-state population protocol models, this result supports the choice of the stronger global fairness condition assumed in the rest of the paper.

Theorem 39. *Let Σ denote the (finite or countably infinite) set of possible input values. A predicate ψ on finite nonempty multisets of elements from Σ is stably computable in the asynchronous message-passing model with the weak fairness condition if and only if ψ is completely determined by the set of input values present in the initial configuration.*

Proof. We describe a protocol in which each agent eventually determines the set of all the inputs that occur in the initial configuration, and therefore the correct value of ψ . The state of each agent is the set consisting of its initial input value together with every value that has occurred in a message it has received. Whenever an agent runs, it sends its current state. Whenever an agent receives a message, it updates its state to be the union of its previous state and the set of values in the message.

Clearly every message sent is a subset of the set of input values in the initial configuration, so there are only finitely many possible messages in each computation. Every message sent by an agent with input value σ contains the element σ , and it sends infinitely many messages, so eventually every agent receives a message containing σ . Thus, the state of every agent eventually consists of the set of values in the input configuration, and each outputs the correct value of ψ .

For the converse, assume that we have a protocol that stably computes a predicate ψ , and let x and x' be any two nonempty multisets of values from Σ such that the same set of values appears in each. Let $n = |x|$ and $n' = |x'|$. Let c_0 and c'_0 be initial configurations corresponding to inputs x and x' , respectively. We construct two executions α and α' starting from c_0 and c'_0 . Let $m_1, m_2, \dots$ be an arbitrary sequence of messages where every possible message appears infinitely often. We construct the executions α and α' in phases, where phase i will ensure that message m_i gets delivered to everyone if that message has been sent enough times. Let c_i and c'_i be the configurations of α and α' at the end of phase i .

Our goal is to prove the following claim: for all $i \geq 0$ and for all $\sigma \in \Sigma$, the state in c_i of each agent with original input σ is the same as the state in c'_i of each agent with original input σ . Assume that we have constructed the first $i - 1$ phases of the two executions so that the claim is satisfied. Suppose we run all processes in lock step from c_{i-1} and c'_{i-1} without delivering any messages. There are two cases.

Case (i): Eventually, after r_i rounds, the run from c_{i-1} will have at least n copies of m_i in transit and, after r'_i rounds, the run from c'_{i-1} will have at least n' copies of m_i in transit. Then, the i th phase of α and α' is constructed by running each agent for $\max(r_i, r'_i)$ rounds without delivering any messages, and then delivering one copy of m_i to every agent. This ensures the claim will be true for c_i and c'_i .

Case (ii): Otherwise, we allow every agent to take one step without delivering any messages. (This clearly satisfies the claim for c_i and c'_i .)

It remains to show that both α and α' satisfy the weak fairness condition, and then it will follow from the claim that $\psi(x) = \psi(x')$. First, notice that every agent takes infinitely many steps in α and α' . If some agent v sends a message m infinitely many times in α or α' , it will also be sent infinitely many times by an agent with the same input value in the other execution (since an agent with a particular input experiences the same sequence of events in both executions). Suppose m is never delivered after phase i to some agent w

in one of the two executions. Eventually, there will be n copies of m in transit in C_j for some $j > i$ and n' copies of m in transit in $C'_{j'}$ for some $j' > i$. Consider the first occurrence of m in the sequence $m_1, m_2, \dots$ that comes after m_j and $m'_{j'}$. During the corresponding phase, m will be delivered to every agent, including w , a contradiction. Thus, α and α' satisfy the weak fairness condition. $\square$

16 Conclusions and Discussion

We have shown that the predicates stably computable by population protocols in the all-pairs communication graph are semilinear, answering the main open question in [AAD⁺04, AAD⁺06]. This result also shows that the model of population protocols with stabilizing inputs, introduced in [AAC⁺05], is equal in computational power to the standard model of population protocols in the all-pairs communication graph, resolving another open question.

Our semilinearity proof is given for an abstract model that includes not only population protocols, but generalizations permitting rules that replace one finite multiset of elements of a configuration with another finite multiset of elements. Thus, even a generalization of population protocols in which rules may add or delete agents or involve interactions between more than two agents will stably compute only semilinear predicates, answering a question in [AAD⁺03].

We have introduced several models of one-way communication in population protocols: queued transmission, immediate and delayed transmission and immediate and delayed observation, and exactly characterized the classes of predicates stably computable in these models in the all-pairs communication network by natural subclasses of the semilinear predicates. These one-way models are more closely related to asynchronous message-passing models, and illuminate the effects of capabilities such as the ability to refuse to accept incoming messages temporarily.

Another natural variant of the population protocol model considers “reversing moves” and requires that a protocol stably compute a predicate despite the possible presence in an execution of a finite number of reversing moves, that is, steps in which a rule $(p, q) \mapsto (p', q')$ is used in reverse: $(p', q') \mapsto (p, q)$. It is not difficult to show that the basic protocols for modulo and threshold predicates are resistant to such reversing moves, and therefore that all the semilinear predicates can be computed by protocols resistant to reversing moves. Thus, requiring such resistance does not reduce the computational power of population protocols in the all-pairs communication graph.

Despite promising results for population protocols in restricted communication graphs [AAC⁺05], much more remains to be understood about their computational power. In particular, the computational power of one-way protocols in restricted communication graphs has not been studied.

17 Acknowledgments

This research was carried out while the third author was an undergraduate at the University of Rochester. The authors would like to thank the reviewers of the extended abstracts on which this paper is based [AAE06b, AAER05] for their thoughtful comments and suggestions.

References

- [AAC⁺05] Dana Angluin, James Aspnes, Melody Chan, Michael J. Fischer, Hong Jiang, and René Peralta. Stably computable properties of network graphs. In Viktor K. Prasanna, Sitharama Iyengar,

- gar, Paul Spirakis, and Matt Welsh, editors, *Distributed Computing in Sensor Systems: First IEEE International Conference, DCOSS 2005, Marina del Rey, CA, USA, June/July, 2005, Proceedings*, volume 3560 of *Lecture Notes in Computer Science*, pages 63–74. Springer-Verlag, June 2005.
- [AAD⁺03] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Urn automata. Technical Report YALEU/DCS/TR-1280, Yale University Department of Computer Science, November 2003.
 - [AAD⁺04] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. In *PODC '04: Proceedings of the Twenty-Third Annual ACM Symposium on Principles of Distributed Computing*, pages 290–299. ACM Press, 2004.
 - [AAD⁺06] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, March 2006.
 - [AAE06a] Dana Angluin, James Aspnes, and David Eisenstat. Fast computation by population protocols with a leader. Technical Report YALEU/DCS/TR-1358, Yale University Department of Computer Science, May 2006. Extended abstract to appear, DISC 2006.
 - [AAE06b] Dana Angluin, James Aspnes, and David Eisenstat. Stably computable predicates are semi-linear. In *Proceedings of the 25th ACM Symposium on Principles of Distributed Computing*, pages 292–299, July 2006.
 - [AAER05] Dana Angluin, James Aspnes, David Eisenstat, and Eric Ruppert. On the power of anonymous one-way communication. In *Ninth International Conference on Principles of Distributed Systems*, pages 307–318, December 2005.
 - [AAFJ05] Dana Angluin, James Aspnes, Michael J. Fischer, and Hong Jiang. Self-stabilizing population protocols. In *Ninth International Conference on Principles of Distributed Systems (pre-proceedings)*, pages 79–90, December 2005.
 - [AGM02] Hagit Attiya, Alla Gorbach, and Shlomo Moran. Computing in totally anonymous asynchronous shared memory systems. *Information and Computation*, 173(2):162–183, March 2002.
 - [Ang80] Dana Angluin. Local and global properties in networks of processors. In *Proceedings of the 12th ACM Symposium on Theory of Computing*, pages 82–93, 1980.
 - [ASS02] James Aspnes, Gauri Shah, and Jatin Shah. Wait-free consensus with infinite arrivals. In *Proceedings of the 34th ACM Symposium on Theory of Computing*, pages 524–533, 2002.
 - [Bai75] Norman T. J. Bailey. *The Mathematical Theory of Infectious Diseases, Second Edition*. Charles Griffin & Co., London and High Wycombe, 1975.
 - [BPSV06] Harry Buhrman, Alessandro Panconesi, Riccardo Silvestri, and Paul Vitanyi. On the importance of having an identity or, is consensus really universal? *Distributed Computing*, 18(3):167–176, 2006.

- [BV99] Paolo Boldi and Sebastiano Vigna. Computing anonymously with arbitrary knowledge. In *Proceedings of the 18th ACM Symposium on Principles of Distributed Computing*, pages 173–179, 1999.
- [BV01] Paolo Boldi and Sebastiano Vigna. An effective characterization of computability in anonymous networks. In *Distributed Computing, 15th International Conference*, pages 33–47, 2001.
- [DF01] Zoë Diamadi and Michael J. Fischer. A simple game for the study of trust in distributed systems. *Wuhan University Journal of Natural Sciences*, 6(1–2):72–82, March 2001. Also appears as Yale Technical Report TR–1207, January 2001.
- [DGFG06] Carole Delporte-Gallet, Hugues Fauconnier, Rachid Guerraoui, and Eric Ruppert. When birds die: Making population protocols fault-tolerant. In *Proceedings of the Second IEEE International Conference on Distributed Computing in Sensor Systems (DCOSS '06)*, pages 51–66, 2006.
- [Dic13] Leonard Eugene Dickson. Finiteness of the odd perfect and primitive abundant numbers with n distinct prime factors. *American Journal of Mathematics*, 35(4):413–422, October 1913.
- [DK65] D. J. Daley and D. G. Kendall. Stochastic rumours. *Journal of the Institute of Mathematics and its Applications*, 1:42–55, 1965.
- [ES94] Ömer Egecioglu and Ambuj K. Singh. Naming symmetric processes using shared variables. *Distributed Computing*, 8(1):19–38, 1994.
- [FR03] Faith Fich and Eric Ruppert. Hundreds of impossibility results for distributed computing. *Distributed Computing*, 16(2-3):121–163, September 2003.
- [Gil92] Daniel T. Gillespie. A rigorous derivation of the chemical master equation. *Physica A*, 188:404–425, 1992.
- [GR05] Rachid Guerraoui and Eric Ruppert. What can be implemented anonymously? In *19th International Symposium on Distributed Computing*, pages 244–259, 2005.
- [Hig52] Graham Higman. Ordering by divisibility in abstract algebras. *Proceedings of the London Mathematical Society*, 3(2):326–336, 1952.
- [HP78] J. Hopcroft and J. Pansiot. On the reachability problem for 5-dimensional vector addition systems. *Theoretical Computer Science*, 8(2):135–159, 1978.
- [IDE04] Oscar H. Ibarra, Zhe Dang, and Omer Egecioglu. Catalytic P systems, semilinear sets, and vector addition systems. *Theor. Comput. Sci.*, 312(2-3):379–399, 2004.
- [JT90] Prasad Jayanti and Sam Toueg. Wakeup under read/write atomicity. In *Distributed Algorithms, 4th International Workshop*, volume 486 of *LNCS*, pages 277–288, 1990.
- [KOPS00] Shay Kutten, Rafail Ostrovsky, and Boaz Patt-Shamir. The Las-Vegas processor identity problem (How and when to be unique). *Journal of Algorithms*, 37(2):468–494, November 2000.
- [Lan02] Serge Lang. *Algebra (Revised Third Edition)*. Springer-Verlag, 2002.

- [Lay92] Steven R. Lay. *Convex Sets and their Applications*. Krieger Publishing Company, 1992.
- [LP90] Richard J. Lipton and Arvin Park. The processor identity problem. *Information Processing Letters*, 36(2):91–94, October 1990.
- [Par66] Rohit J. Parikh. On context-free languages. *J. ACM*, 13(4):570–581, 1966.
- [PPTV98] Alessandro Panconesi, Marina Papatriantafylou, Philippas Tsigas, and Paul Vitányi. Randomized naming using wait-free shared variables. *Distributed Computing*, 11(3):113–124, August 1998.
- [Pre29] Mojzesz Presburger. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In *Comptes-Rendus du I Congrès de Mathématiciens des Pays Slaves*, pages 92–101, Warszawa, 1929.
- [Sak99] Naoshi Sakamoto. Comparison of initial conditions for distributed algorithms on anonymous networks. In *Proc. 18th ACM Symposium on Principles of Distributed Computing*, pages 173–179, 1999.
- [Ten90] Shang-Hua Teng. Space efficient processor identity protocol. *Information Processing Letters*, 34(3):147–154, April 1990.